

04 – Advanced SQL

DML

Advanced queries for your Database

Jérôme Landré – jerome.landre@ju.se

OUTLINE

- 1) The AdventureWorks2019 database
- 2) Methodology to create a query
- 3) SELECT queries without joining
- 4) Joining tables
- 5) Different types of JOINS

TIPS AND TRICKS

- Be **positive** means:
 - be **organized** (to avoid stress) and
 - be **flexible** (react to unpredicted events in a positive way).
- **Respect deadlines**: in your schedule, note the important **deadlines** and add a note/recall **one week before** to be sure not to forget!
- React **calmly** to unexpected events: define **priorities** and **organize** your **time**.

TIPS AND TRICKS

- Your **time** is very **precious**! Don't waste it...

- Soprano, “Mon précieux” (“my precious”):

<https://www.youtube.com/watch?v=OVmfGb8XKSg>

- Lyrics (English and Swedish translation):

<https://lyricstranslate.com/en/soprano-mon-precieux-english>

1) THE ADVENTUREWORKS DATABASE

- The AdventureWorks database is the “official” database example in MS SQL Server
- It modelizes a company producing and selling products with many different services (**SCHEMAS**) and tables
- It contains a lot of information per table and allows to **study the CRUD** and **run queries** on “real” data
- There are many different versions available, in this course, we will use AdventureWorks2019

<https://learn.microsoft.com/en-us/sql/samples/adventureworks-install-configure?view=sql-server-ver16&tabs=ssms>

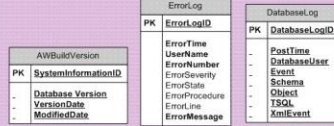
AdventureWorks 2008 OLTP Schema

Rev 10.00.0009

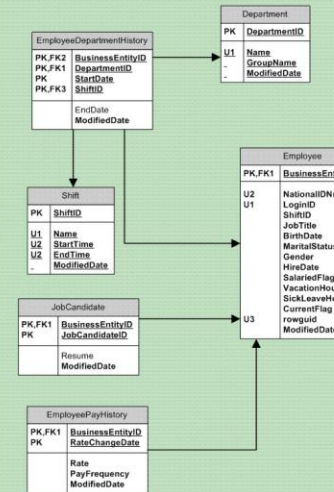
Best Print Results if:
11X17 paper
Landscape
Fit to 1 sheet

Samples and Sample Databases at
<http://CodePlex.com/SqlServerSamples>

dbo



HumanResources



Schemas

Sales

Purchasing

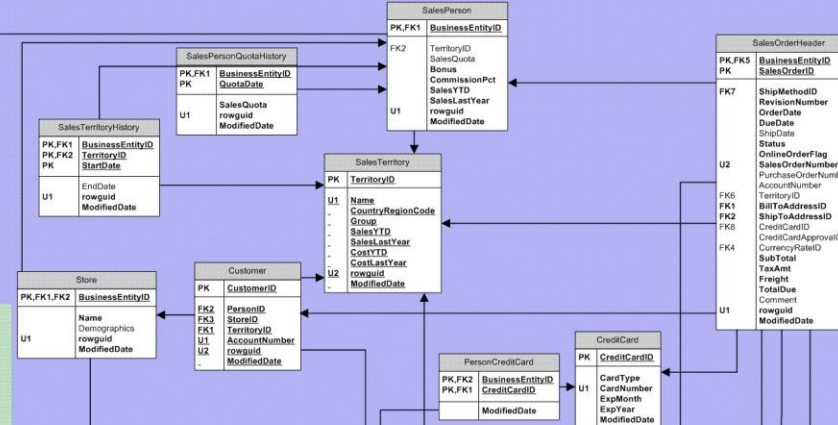
Person

Production

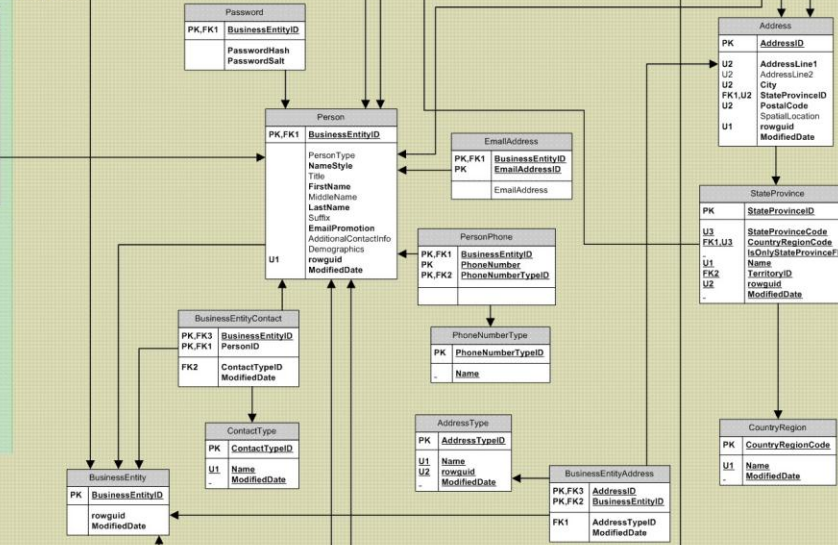
HumanResources

dbo

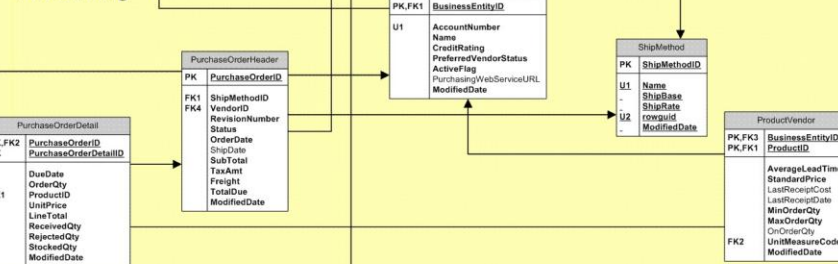
Sales



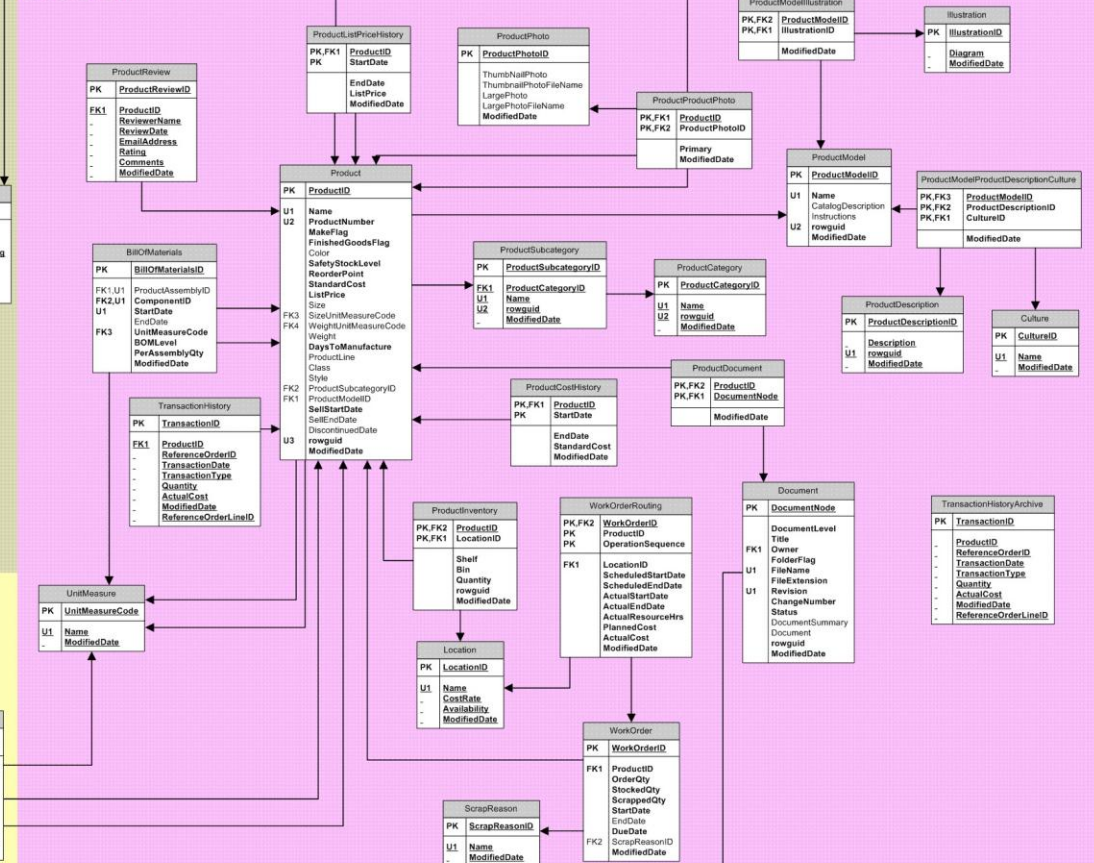
Person



Purchasing



Production



Schemas

Sales

Purchasing

Person

Production

HumanResources

dbo

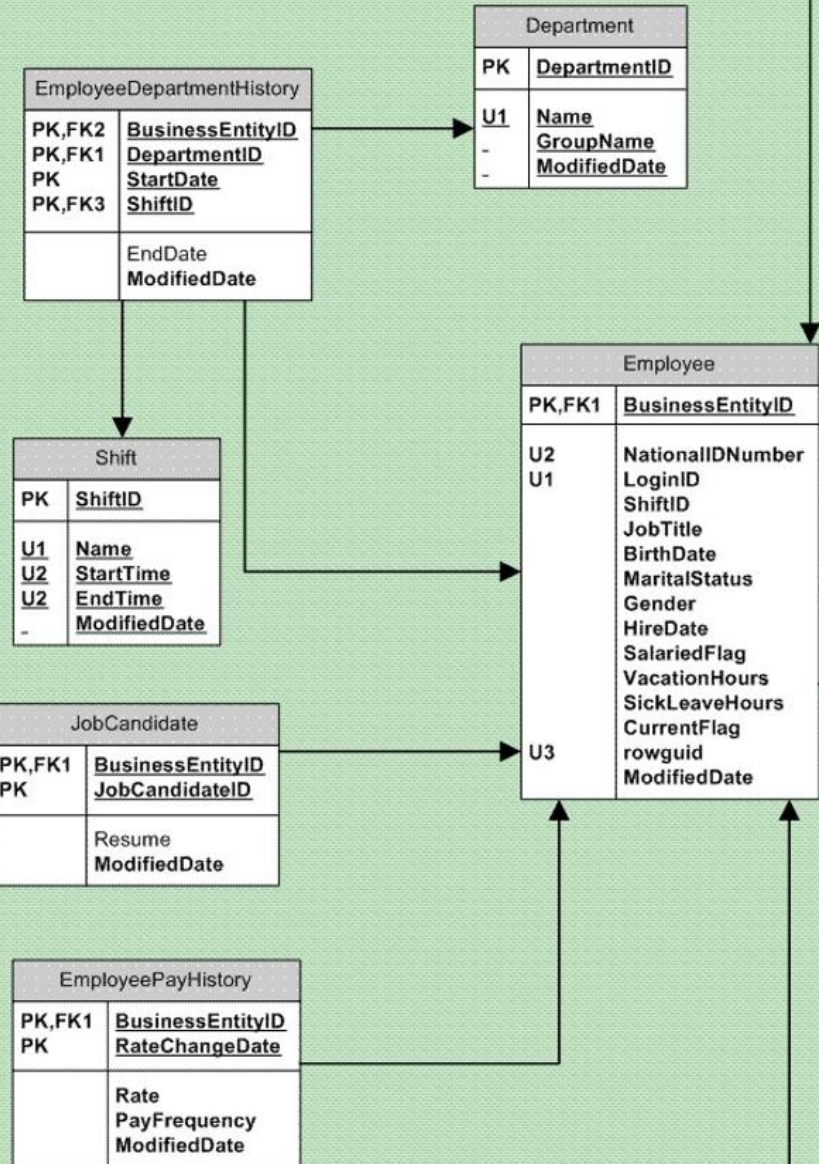
dbo

AWBuildVersion	
PK	<u>SystemInformationID</u>
-	<u>Database Version</u>
-	<u>VersionDate</u>
-	<u>ModifiedDate</u>

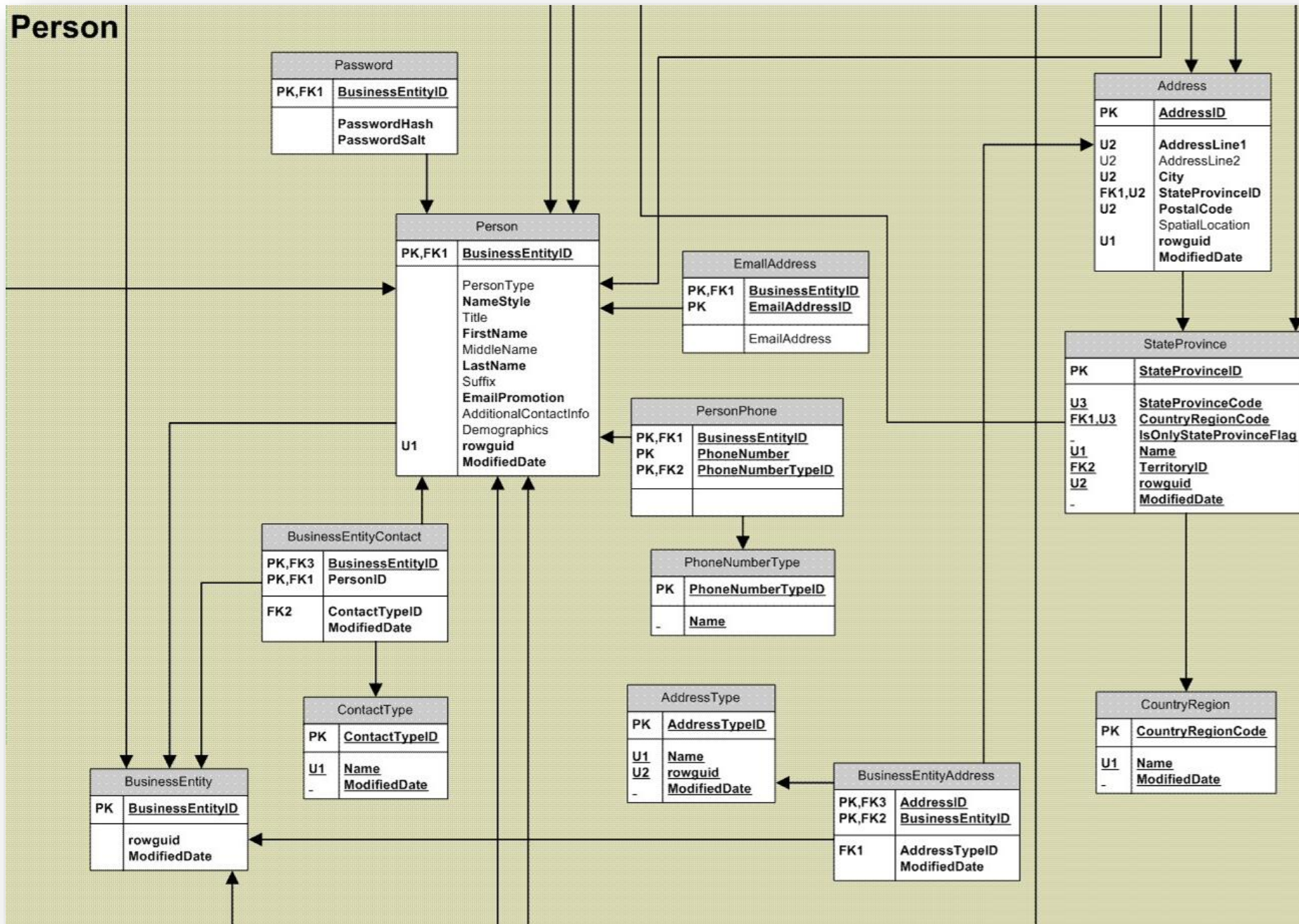
ErrorLog	
PK	<u>ErrorLogID</u>
	ErrorTime
	UserName
	ErrorNumber
	ErrorSeverity
	ErrorState
	ErrorProcedure
	ErrorLine
	ErrorMessage

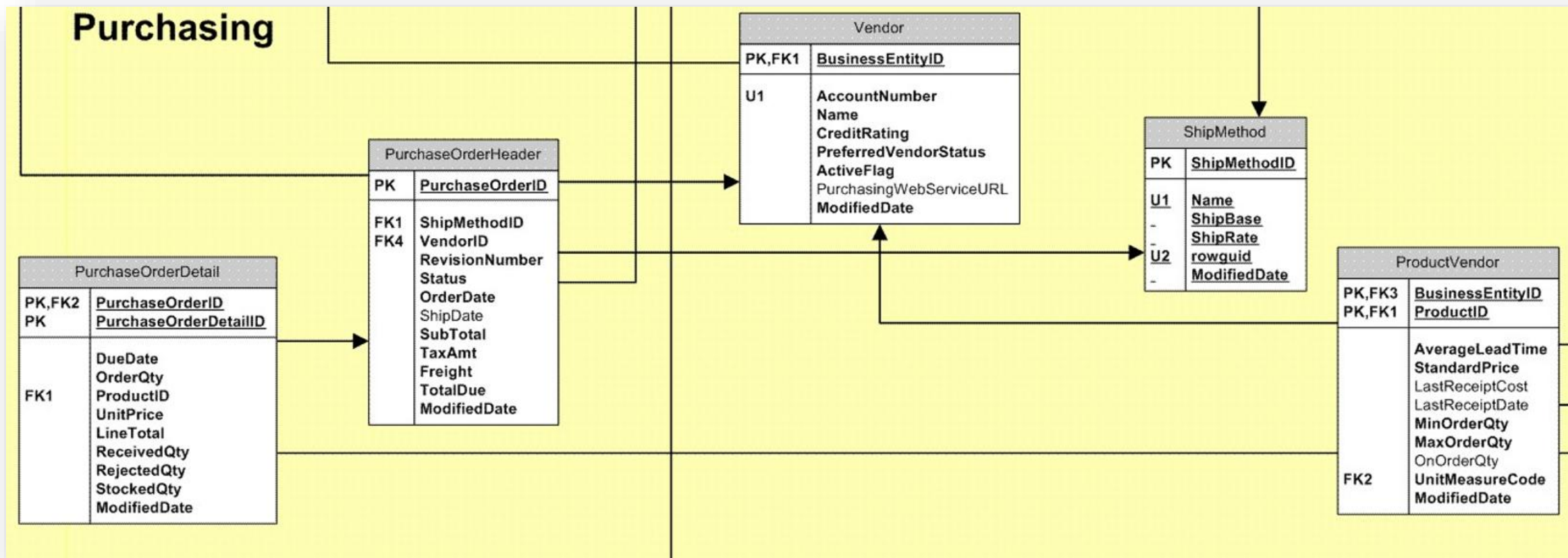
DatabaseLog	
PK	<u>DatabaseLogID</u>
-	<u>PostTime</u>
-	<u>DatabaseUser</u>
-	<u>Event</u>
-	<u>Schema</u>
-	<u>Object</u>
-	<u>TSQL</u>
-	<u>XmlEvent</u>

HumanResources

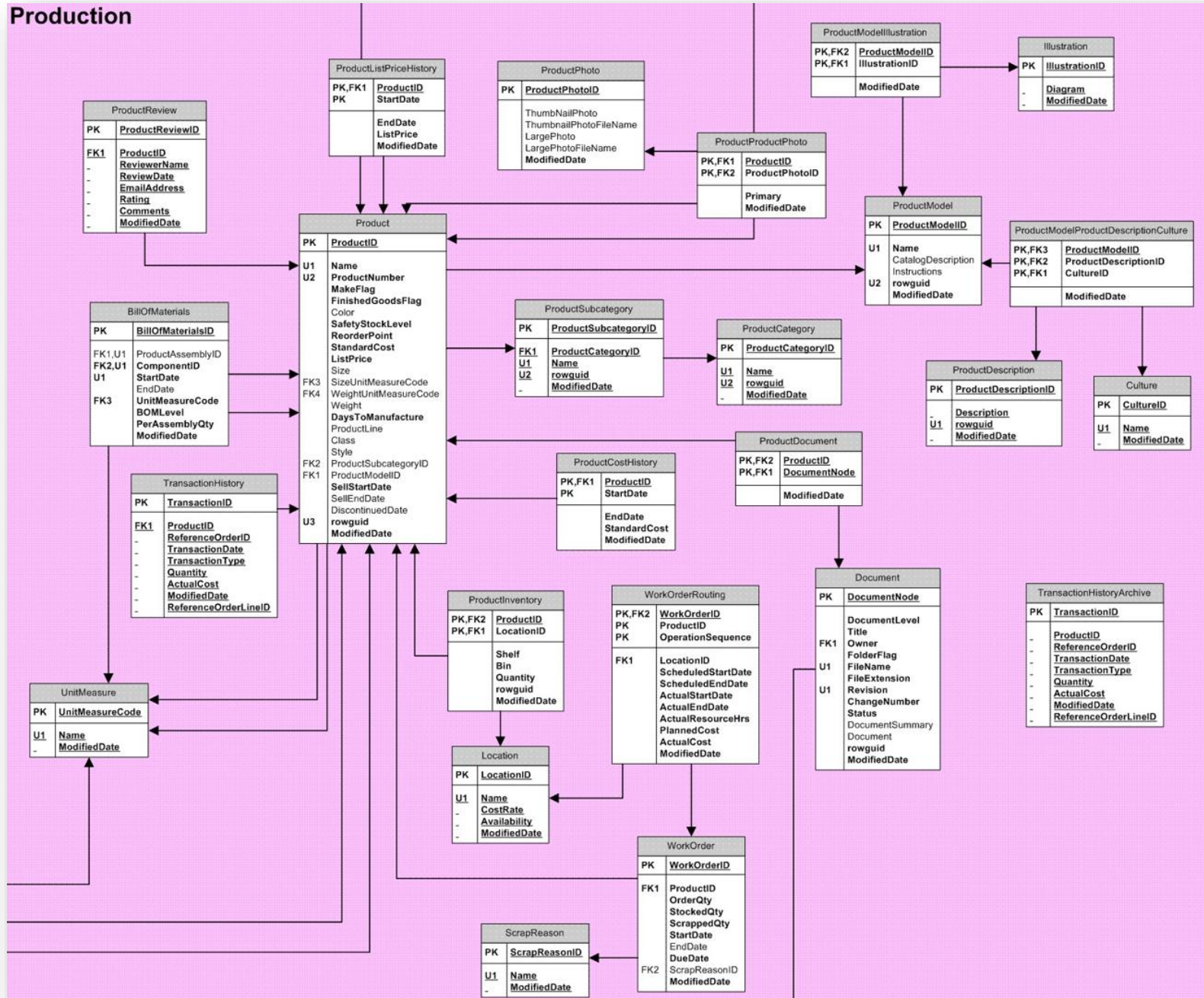


Person

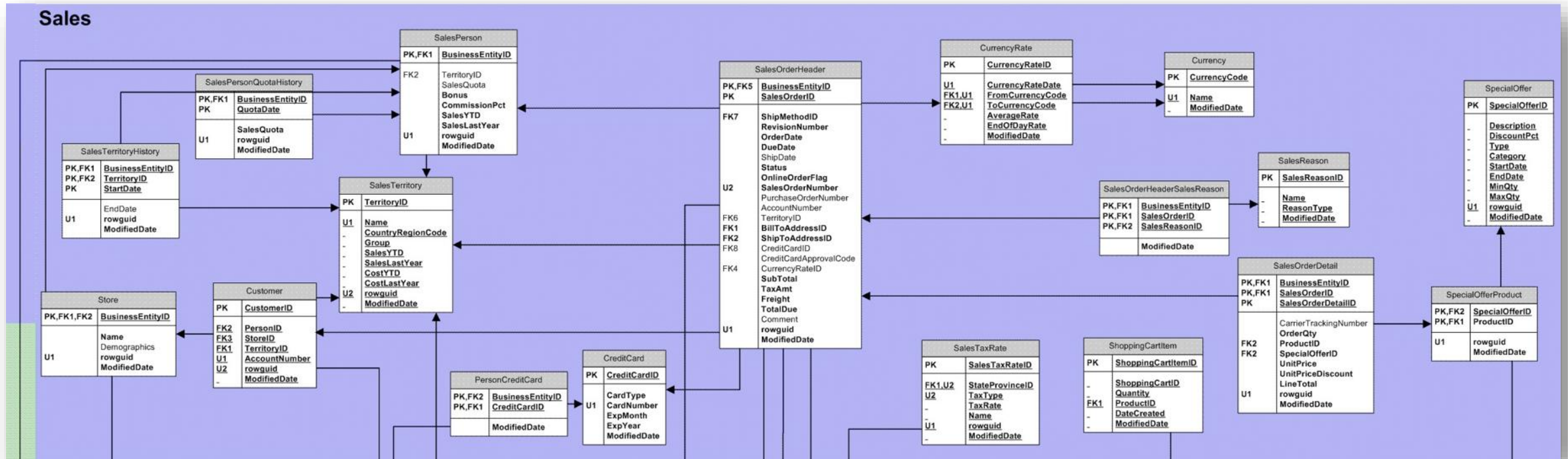




Production



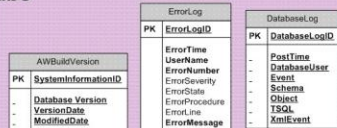
Sales



Best Print Results if:
11X17 paper
Landscape
Fit to 1 sheet

Samples and Sample Databases at
<http://CodePlex.com/SqlServerSamples>

dbo



```

    erDiagram
        Department ||--}| EmployeeDepartmentHistory : "1"
        EmployeeDepartmentHistory ||--}| Shift : "1"
        Shift ||--}| JobCandidate : "1"
        JobCandidate ||--}| Employee : "1"
        Employee ||--}| EmployeePayHistory : "1"
        Employee ||--}| JobCandidate : "1"

        Department {
            int PK_DepartmentID PK
        }

        EmployeeDepartmentHistory {
            int FK2_BusinessEntityID FK
            int PK_DepartmentID PK
            int PK_ShiftID PK
            int FK_EndDate FK
            int ModifiedDate
        }

        Shift {
            int PK_ShiftID PK
            int FK_U1 FK
            int FK_U2 FK
            int FK_U3 FK
            int Name
            int StartTime
            int EndTime
            int ModifiedDate
        }

        JobCandidate {
            int FK_FK1_BusinessEntityID FK
            int FK_JobCandidateID FK
            int Resume
            int ModifiedDate
        }

        Employee {
            int PK_FK1_BusinessEntityID PK
            int FK_U2 FK
            int FK_U3 FK
            int NationalIDNumber
            int LoginID
            int ShiftID
            int JobTitle
            int BirthDate
            int Gender
            int HireDate
            int SalariedFlag
            int VacationHours
            int SickLeaveHours
            int CurrentFlag
            int rowguid
            int ModifiedDate
        }

        EmployeePayHistory {
            int FK_FK1_BusinessEntityID FK
            int FK_RateChangeDate FK
            int Rate
            int PayFrequency
            int ModifiedDate
        }
  
```

Sales

Purchasing

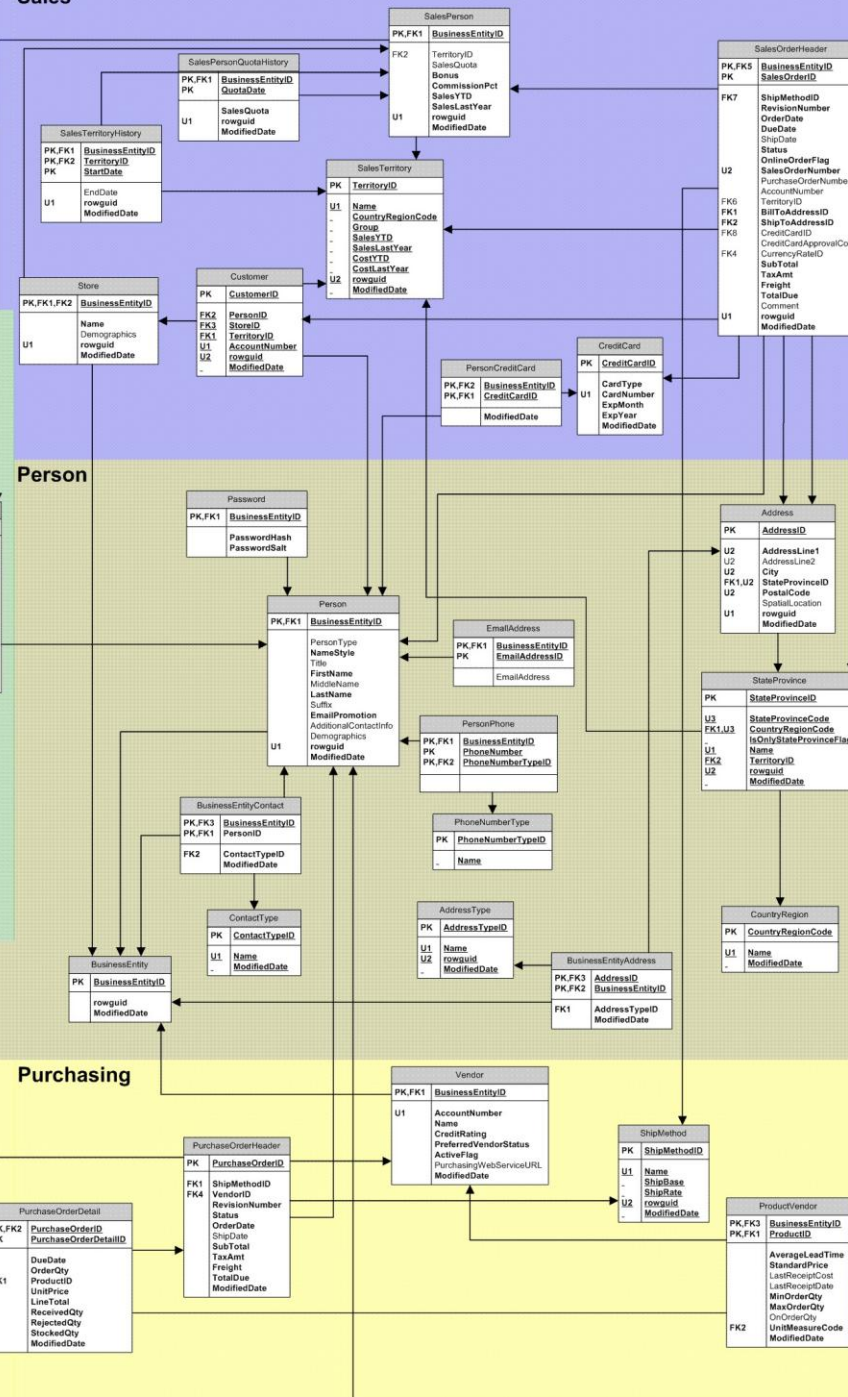
Person

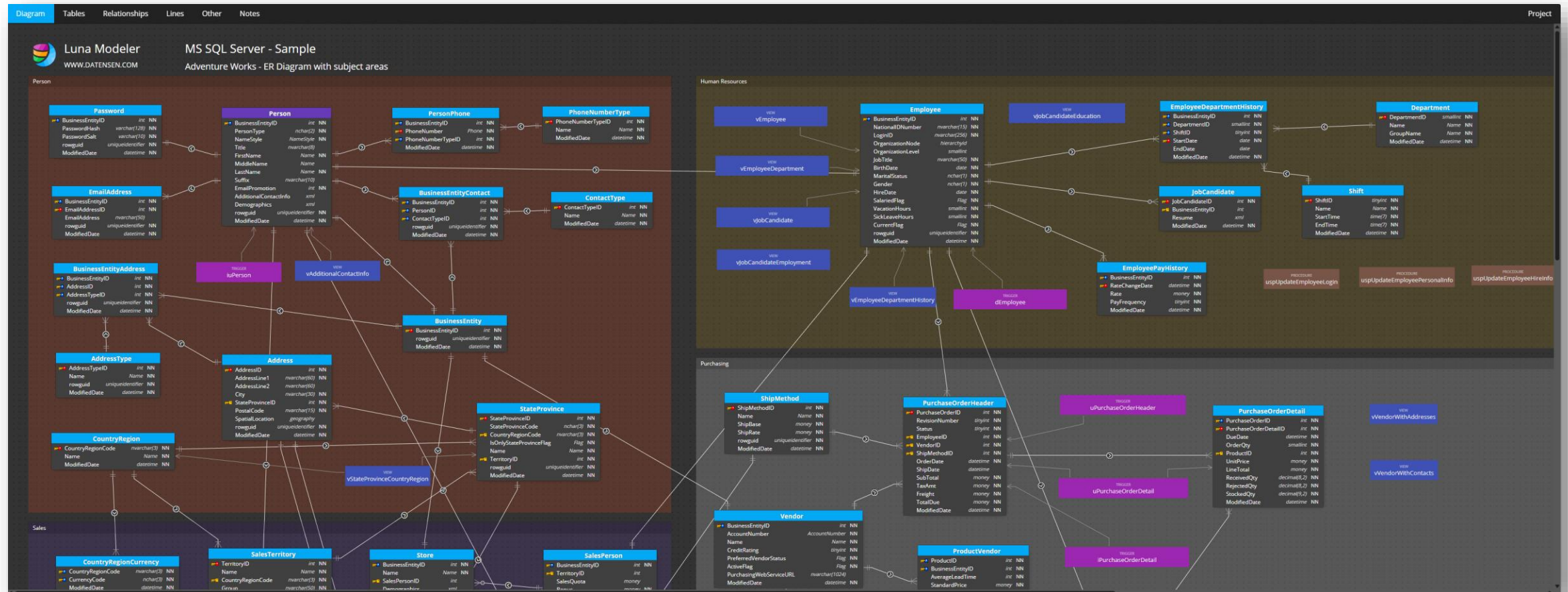
Production

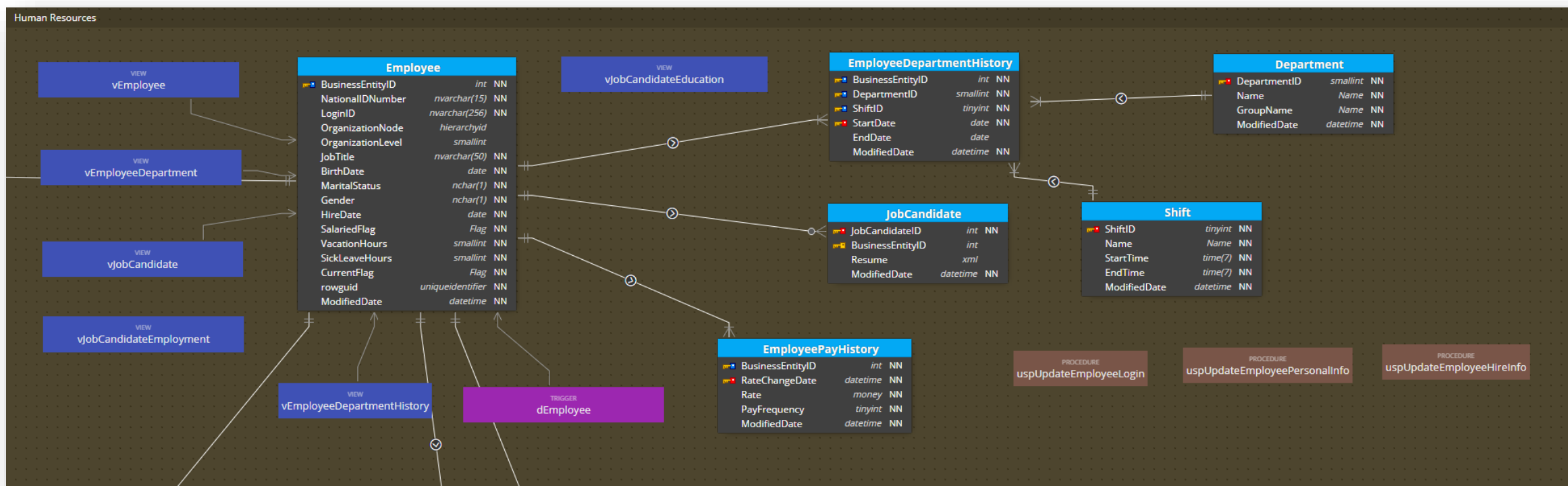
HumanResources

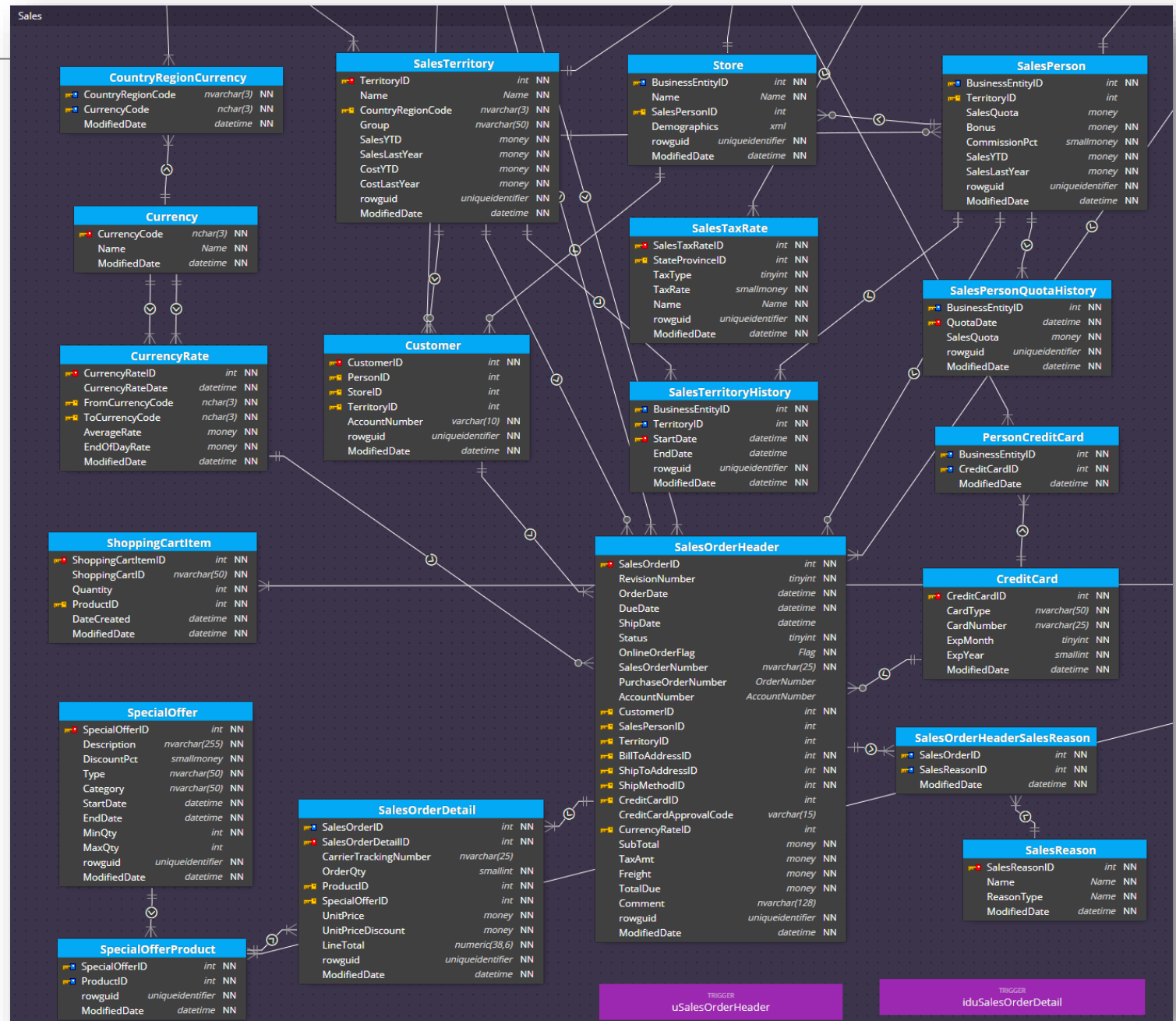
dbo

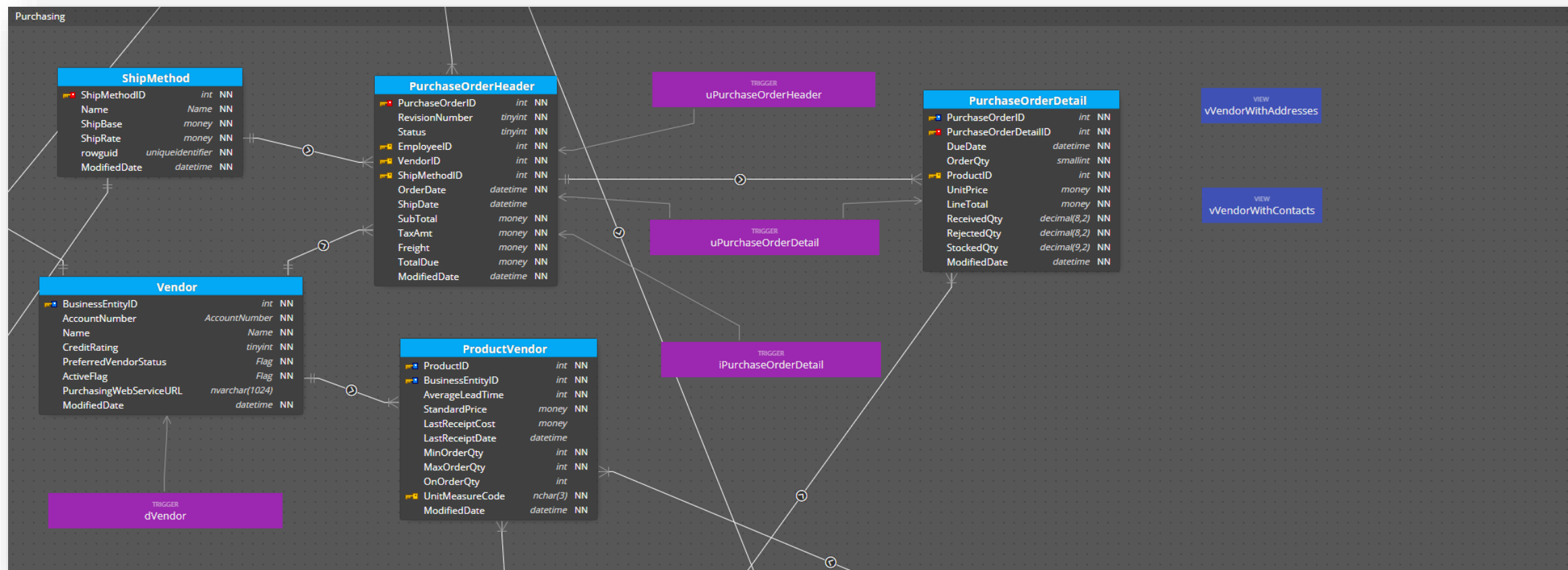
Person

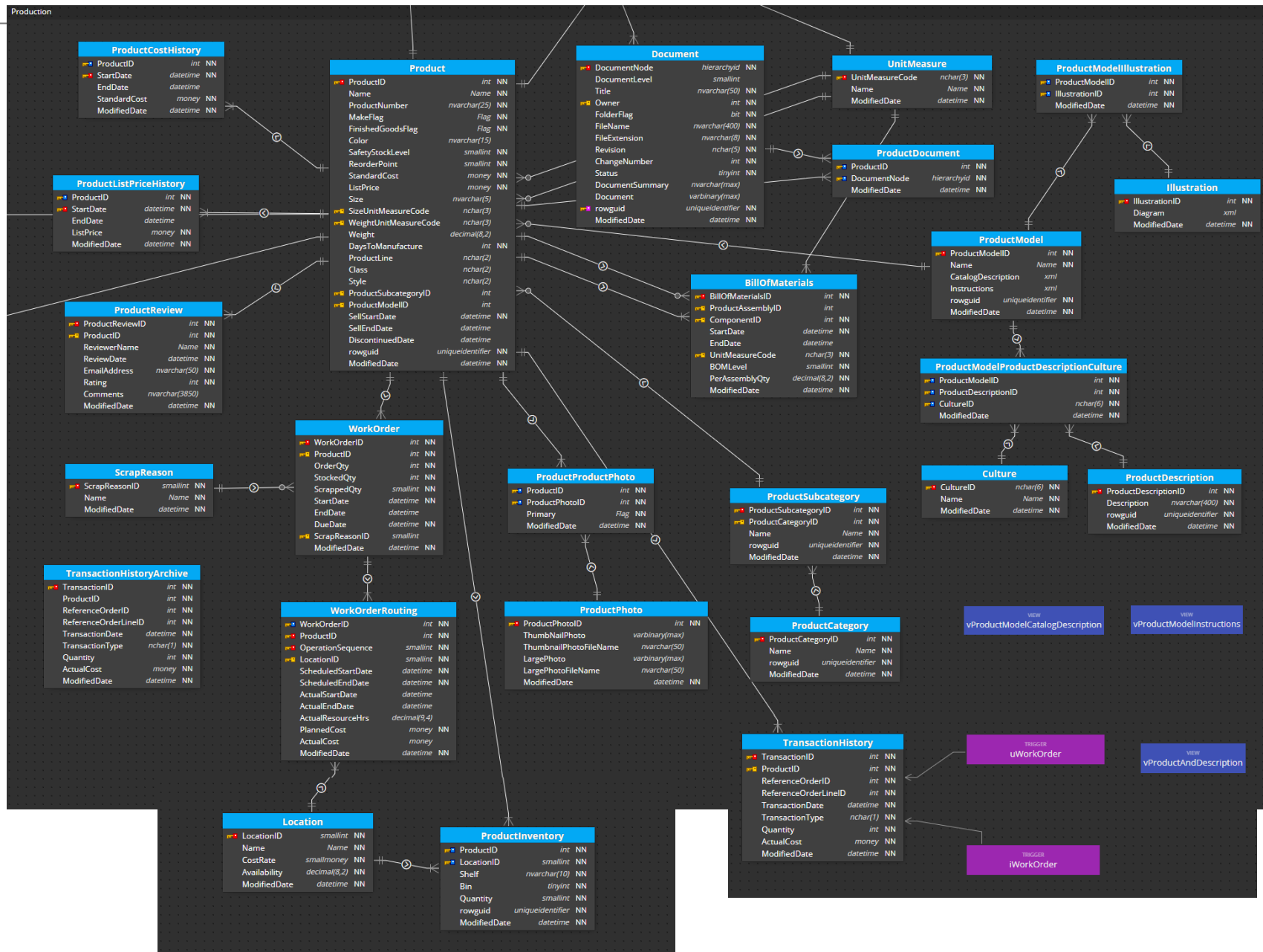
[illegible]











AWBuildVersion		
SystemInformationID	tinyint	NN
Database Version	nvarchar(25)	NN
VersionDate	datetime	NN
ModifiedDate	datetime	NN

DatabaseLog		
DatabaseLogID	int	NN
PostTime	datetime	NN
DatabaseUser	sysname	NN
Event	sysname	NN
Schema	sysname	
Object	sysname	
TSQL	nvarchar(max)	NN
XmlEvent	xml	NN

ErrorLog		
ErrorLogID	int	NN
ErrorTime	datetime	NN
UserName	sysname	NN
ErrorNumber	int	NN
ErrorSeverity	int	
ErrorState	int	
ErrorProcedure	nvarchar(126)	
ErrorLine	int	
ErrorMessage	nvarchar(4000)	NN

USER DEFINED TYPE
Flag

USER DEFINED TYPE
Phone

USER DEFINED TYPE
NameStyle

USER DEFINED TYPE
AccountNumber

USER DEFINED TYPE
Name

USER DEFINED TYPE
OrderNumber

FUNCTION
ufnGetAccountingEndDate

FUNCTION
ufnGetAccountingStartDate

PROCEDURE
uspGetBillOfMaterials

PROCEDURE
uspGetEmployeeManagers

FUNCTION
ufnGetContactInformation

FUNCTION
ufnGetStock

PROCEDURE
uspGetManagerEmployees

PROCEDURE
uspGetWhereUsedProductID

FUNCTION
ufnLeadingZeros

FUNCTION
ufnGetSalesOrderStatusText

PROCEDURE
uspPrintError

PROCEDURE
uspLogError

FUNCTION
ufnGetPurchaseOrderStatusText

FUNCTION
ufnGetProductStandardCost

FUNCTION
ufnGetProductListPrice

FUNCTION
ufnGetDocumentStatusText

PROCEDURE
uspSearchCandidateResumes

FUNCTION
ufnGetProductDealerPrice

2) METHODOLOGY TO CREATE A QUERY

- In the CRUD, **Data Manipulation Language (DML)**, we use **SELECT** to retrieve information from the tables
- Let's play with SELECT queries!
- **RECALL:** you can **ONLY** connect to **JTH MSSQL Server** from the computers in the computer rooms **at JTH** (only running Windows)!

Person

```
erDiagram
    Person ||--o{ BusinessEntity : "has"
    Person ||--o{ Address : "has"
    Person ||--o{ StateProvince : "has"
    Person ||--o{ CountryRegion : "has"
    Person ||--o{ BusinessEntityContact : "has"
    Person ||--o{ ContactType : "has"
    Person ||--o{ BusinessEntityAddress : "has"
    Person ||--o{ EmailAddress : "has"
    Person ||--o{ PersonPhone : "has"
    Person ||--o{ PhoneNumberType : "has"

    Person {
        string BusinessEntityID PK,FK1
        string PersonType
        string NameStyle
        string Title
        string FirstName
        string MiddleName
        string LastName
        string Suffix
        string EmailPromotion
        string AdditionalContactInfo
        string Demographics
        string rowguid PK
        string ModifiedDate
    }

    BusinessEntity {
        string BusinessEntityID PK
        string rowguid
        string ModifiedDate
    }

    Address {
        string AddressID PK
        string AddressLine1
        string AddressLine2
        string City
        string StateProvinceID FK1,U2
        string PostalCode
        string SpatialLocation
        string rowguid
        string ModifiedDate
    }

    StateProvince {
        string StateProvinceID PK
        string StateProvinceCode
        string CountryRegionCode FK1,U3
        string IsOnlyStateProvinceFlag
        string Name
        string TerritoryID
        string rowguid
        string ModifiedDate
    }

    CountryRegion {
        string CountryRegionCode PK
        string Name
        string ModifiedDate
    }

    BusinessEntityContact {
        string BusinessEntityID PK,FK3
        string PersonID PK,FK1
        string ContactTypeID FK2
        string ModifiedDate
    }

    ContactType {
        string ContactTypeID PK
        string Name
        string ModifiedDate
    }

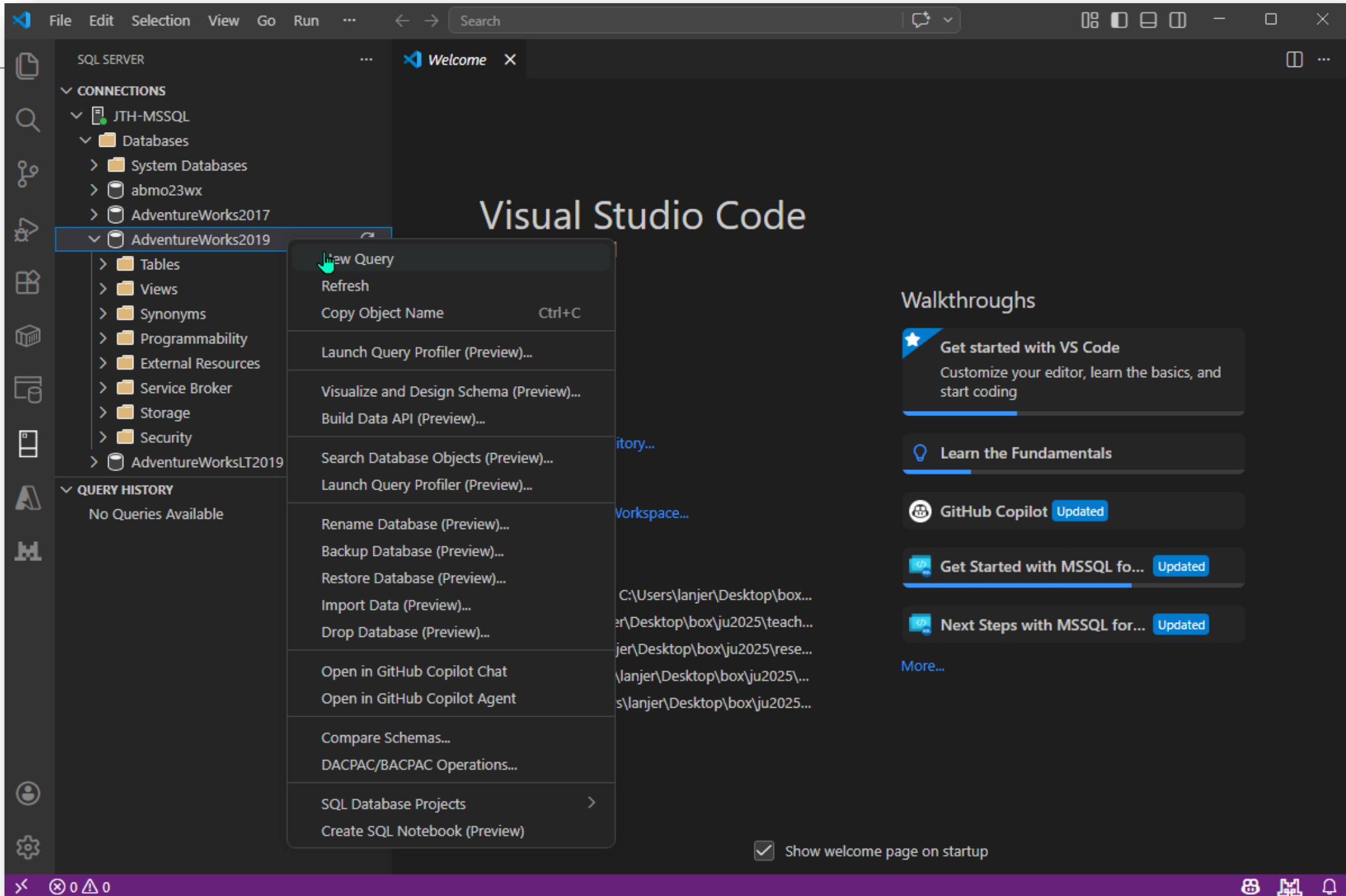
    BusinessEntityAddress {
        string AddressID PK,FK3
        string BusinessEntityID PK,FK2
        string AddressTypeID FK1
        string ModifiedDate
    }

    EmailAddress {
        string BusinessEntityID PK,FK1
        string EmailAddressID PK
        string EmailAddress
    }

    PersonPhone {
        string BusinessEntityID PK,FK1
        string PhoneNumber PK
        string PhoneNumberTypeID PK,FK2
    }

    PhoneNumberType {
        string PhoneNumberTypeID PK
        string Name
    }
```

The diagram illustrates the relationships between various database tables. The **Person** table is the central entity, linked to **BusinessEntity**, **Address**, **StateProvince**, **CountryRegion**, **BusinessEntityContact**, **ContactType**, **BusinessEntityAddress**, **EmailAddress**, **PersonPhone**, and **PhoneNumberType**. The **BusinessEntity** table is linked to **BusinessEntityContact** and **BusinessEntityAddress**. The **Address** table is linked to **StateProvince** and **CountryRegion**. The **BusinessEntityContact** table is linked to **ContactType**. The **BusinessEntityAddress** table is linked to **AddressType**. The **EmailAddress** table is linked to **Person**. The **PersonPhone** table is linked to **PhoneNumberType**. The **StateProvince** table is linked to **CountryRegion**. The **CountryRegion** table is linked to **StateProvince**. The **AddressType** table is linked to **BusinessEntityAddress**. The **BusinessEntity** table is linked to **BusinessEntityContact** and **BusinessEntityAddress**. The **BusinessEntityContact** table is linked to **ContactType**. The **ContactType** table is linked to **BusinessEntityContact**. The **BusinessEntityAddress** table is linked to **AddressType**. The **AddressType** table is linked to **BusinessEntityAddress**. The **EmailAddress** table is linked to **Person**. The **PersonPhone** table is linked to **PhoneNumberType**. The **PhoneNumberType** table is linked to **PersonPhone**. The **StateProvince** table is linked to **CountryRegion**. The **CountryRegion** table is linked to **StateProvince**. The **Address** table is linked to **StateProvince** and **CountryRegion**. The **Person** table is linked to **BusinessEntity**, **Address**, **StateProvince**, **CountryRegion**, **BusinessEntityContact**, **ContactType**, **BusinessEntityAddress**, **EmailAddress**, **PersonPhone**, and **PhoneNumberType**.



SQL SERVER

CONNECTIONS

- JTH-MSSQL
 - Databases
 - System Databases
 - abmo23wx
 - AdventureWorks2017
 - AdventureWorks2019
 - Tables
 - Views
 - Synonyms
 - Programmability
 - External Resources
 - Service Broker
 - Storage
 - Security
 - AdventureWorksLT2019

QUERY HISTORY

No Queries Available

USE AdventureWorks2019; Untitled-1

```
1 jthstudsql.win.hj.se | AdventureWorks2019
2 USE AdventureWorks2019;
3
4 SELECT * FROM Person.Person;
```

Execute Query (Ctrl+Shift+E)

Ln 4, Col 1 Spaces: 4 UTF-8 CRLF { } SQL SQLCMD: Off MSSQL jthstudsql.win.hj.se AdventureWorks2019 Prettier

1

SQL SERVER

CONNECTIONS

JTH-MSSQL

Databases

System Databases

abmo23wx

AdventureWorks2017

AdventureWorks20...

Tables

Views

Synonyms

Programmability

External Resources

Service Broker

Storage

Security

AdventureWorksLT2019

QUERY HISTORY

No Queries Available

USE AdventureWorks2019; Untitled-1

jthstudsql.win.hj.se | AdventureWorks2019

USE AdventureWorks2019;

SELECT * FROM Person.Person;

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

QUERY RESULTS

AZURE

Results (1)

Messages

Open in New Tab

	BusinessEntityID		PersonType		NameStyle		Title		FirstName		MiddleName		Las
1	1		EM		0		NULL		Ken		J		S
2	2		EM		0		NULL		Terri		Lee		D
3	3		EM		0		NULL		Roberto		NULL		Ti
4	4		EM		0		NULL		Rob		NULL		V
5	5		EM		0		Ms.		Gail		A		E
6	6		EM		0		Mr.		Jossef		H		G
7	7		EM		0		NULL		Dylan		A		M
8	8		EM		0		NULL		Diane		L		M
9	9		EM		0		NULL		Gigi		N		M
10	10		EM		0		NULL		Michael		NULL		R
11	11		EM		0		NULL		Ovidiu		V		C

<

0

0

SQL

SQLite: No database

MSSQL

SQLCMD: Off

19972 rows affected

00:00:01.143

jthstudsql.win.hj.se

AdventureWorks2019

Query executed

Prettier

SQL SERVER

CONNECT... +

JTH-MSSQL

Databases

AdventureWorks2019

Tables

- Person.CountryRegi...
- Person.EmailAddress
- Person.Password
- Person.Person

Columns

- BusinessEntityID ...
- PersonType (nch...
- NameStyle (Nam...
- Title (nvarchar(8),...
- FirstName (Nam...
- MiddleName (Na...
- LastName (Name...
- Suffix (nvarchar(1...
- EmailPromotion (...
- AdditionalContac...
- Demographics (I...
- rowguid (uniquei...
- ModifiedDate (d...

Keys

Constraints

QUERY HISTORY

No Queries Available

USE AdventureWorks2019; Untitled-1

```
jthstuds.sql.win.hj.se | AdventureWorks2019
1 USE AdventureWorks2019;
2
3 SELECT * FROM Person.Person;
4
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS AZURE

Results (1) Messages

Open in New Tab

	BusinessEntityID	PersonType	NameStyle	Title	FirstName	MiddleName	Las
1	1	EM	0	NULL	Ken	J	S
2	2	EM	0	NULL	Terri	Lee	C
3	3	EM	0	NULL	Roberto	NULL	Ti
4	4	EM	0	NULL	Rob	NULL	V
5	5	EM	0	Ms.	Gail	A	E
6	6	EM	0	Mr.	Jossef	H	G
7	7	EM	0	NULL	Dylan	A	M
8	8	EM	0	NULL	Diane	L	M
9	9	EM	0	NULL	Gigi	N	M
10	10	EM	0	NULL	Michael	NULL	R
11	11	EM	0	NULL	Ovidiu	V	C

SQLite: No database MSSQL SQLCMD: Off 19972 rows affected 00:00:01.143 ✓ jthstuds.sql.win.hj.se AdventureWorks2019 Query executed Prettier

SQL SERVER

CONNECTIONS

- JTH-MSSQL
 - Databases
 - AdventureWorks2019
 - Tables
 - Person.CountryRegi...
 - Person.EmailAddress
 - Person.Password
 - Person.Person
 - Columns
 - BusinessEntityID ...
 - PersonType (nch...
 - NameStyle (Nam...
 - Title (nvarchar(8),...
 - FirstName (Nam...
 - MiddleName (Na...
 - LastName (Name...
 - Suffix (nvarchar(1...
 - EmailPromotion (...
 - AdditionalContac...
 - Demographics (I...
 - rowguid (uniquei...
 - ModifiedDate (d...
 - Keys
 - Constraints

QUERY HISTORY

No Queries Available

USE AdventureWorks2019; Untitled-1

jthstudsq1.win.hj.se | AdventureWorks2019

```
1 USE AdventureWorks2019;
2
3 SELECT * FROM Person.Person;
4
5 SELECT FirstName AS fn, LastName AS ln, ModifiedDate AS ModifDate FROM Person.Person;
6
7
8
9
```

Execute Query (Ctrl+Shift+E)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS AZURE

Results (1) Messages

Open in New Tab

	BusinessEntityID	PersonType	NameStyle	Title	FirstName	MiddleName	Las
1	1	EM	0	NULL	Ken	J	S
2	2	EM	0	NULL	Terri	Lee	C
3	3	EM	0	NULL	Roberto	NULL	Ti
4	4	EM	0	NULL	Rob	NULL	V
5	5	EM	0	Ms.	Gail	A	E
6	6	EM	0	Mr.	Jossef	H	G
7	7	EM	0	NULL	Dylan	A	M
8	8	EM	0	NULL	Diane	L	M
9	9	EM	0	NULL	Gigi	N	M
10	10	EM	0	NULL	Michael	NULL	R
11	11	EM	0	NULL	Ovidiu	V	C

0 0 0 {} SQL SQLite: No database MSSQL SQLCMD: Off 19972 rows affected 00:00:01.143 ✓ jthstudsq1.win.hj.se AdventureWorks2019 Query executed Prettier

SQL SERVER

CONNECTIONS

- JTH-MSSQL
 - Databases
 - AdventureWorks2019
 - Tables
 - Person.CountryRegi...
 - Person.EmailAddress
 - Person.Password
 - Person.Person
 - Columns
 - BusinessEntityID ...
 - PersonType (nch...
 - NameStyle (Nam...
 - Title (nvarchar(8),...
 - FirstName (Nam...
 - MiddleName (Na...
 - LastName (Name...
 - Suffix (nvarchar(1...
 - EmailPromotion (...
 - AdditionalContac...
 - Demographics (I...
 - rowguid (uniquei...
 - ModifiedDate (d...
 - Keys
 - Constraints

QUERY HISTORY

No Queries Available

USE AdventureWorks2019; Untitled-1

jthstudsql.win.hj.se | AdventureWorks2019

```
1 USE AdventureWorks2019;
2
3 SELECT * FROM Person.Person;
4
5 SELECT FirstName AS fn, LastName AS ln, ModifiedDate AS ModifDate FROM Person.Person;
```

Execute Query (Ctrl+Shift+E)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS AZURE

Results (1) Messages

	fn	ln	ModifDate
1	Ken	Sánchez	2009-01-07 00:00:00.000
2	Terri	Duffy	2008-01-24 00:00:00.000
3	Roberto	Tamburello	2007-11-04 00:00:00.000
4	Rob	Walters	2007-11-28 00:00:00.000
5	Gail	Erickson	2007-12-30 00:00:00.000
6	Jossef	Goldberg	2013-12-16 00:00:00.000
7	Dylan	Miller	2009-02-01 00:00:00.000
8	Diane	Margheim	2008-12-22 00:00:00.000
9	Gigi	Matthew	2009-01-09 00:00:00.000
10	Michael	Raheem	2009-04-26 00:00:00.000
11	Ovidiu	Cracium	2010-11-28 00:00:00.000

Open in New Tab

0 UTF-8 CRLF {} SQL SQLite: No database MSSQL SQLCMD: Off 19972 rows affected 00:00:00.159 ✓ jthstudsql.win.hj.se AdventureWorks2019 Prettier

2) METHODOLOGY TO CREATE A QUERY

- Locate **all the tables necessary** to find the information:
 - If one table only, **SELECT** the correct **COLUMNS** with a **WHERE** clause if necessary, for instance, **find all the persons with firstname “Jerome” in the database:**

```
SELECT firstname, lastname FROM Person.Person  
WHERE firstname='Jerome';
```

- If two or more tables, use a **JOIN** to mix tables linked by a relation.

3) SELECT QUERIES WITHOUT JOINING

- Find all the persons with lastname starting by “LAN” in the database:

```
SELECT firstname AS fn, lastname AS ln
FROM Person.Person
WHERE lastname LIKE 'Lan%';
```

Results (1)		Messages	
	fn ↑↓ ▾	ln ↑↓ ▾	
1	Andrew	Lan	
2	Eric	Lang	

- Find all the persons with firstname starting by “Mar” and lastname starting by “Bak” in the database:

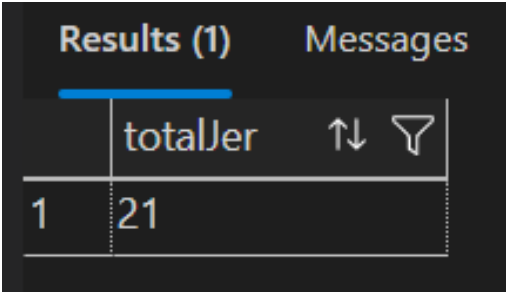
```
SELECT firstname AS fn, lastname AS ln
FROM Person.Person
WHERE firstname LIKE 'Mar%' AND
lastname LIKE 'Bak%';
```

Results (1)		Messages	
	fn ↑↓ ▾	ln ↑↓ ▾	
1	Marcus	Baker	
2	Maria	Baker	
3	Mary	Baker	
4	Mary	Baker	

3) SELECT QUERIES WITHOUT JOINING

- Count all the persons with firstname “Jerome” in the database:

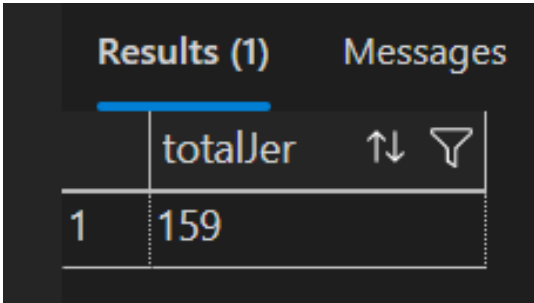
```
SELECT COUNT(*) AS totalJer  
FROM Person.Person  
WHERE firstname='Jerome';
```



Results (1)		Messages
	totalJer	↕ ⚙
1	21	

- Count all the persons with firstname starting by “Jer” in the database:

```
SELECT COUNT(*) AS totalJer  
FROM Person.Person  
WHERE firstname LIKE 'Jer%';
```



Results (1)		Messages
	totalJer	↕ ⚙
1	159	

3) SELECT QUERIES WITHOUT JOINING

- Order the results on a field:

```
SELECT * FROM Person.Person ORDER BY lastname DESC; -- ASC
```

- Math computation:

+, -, /, *

SQRT(a), POWER(base, exponent)

SIN(), COS(), EXP(), LOG()

PI(), ...

- Optimal values:

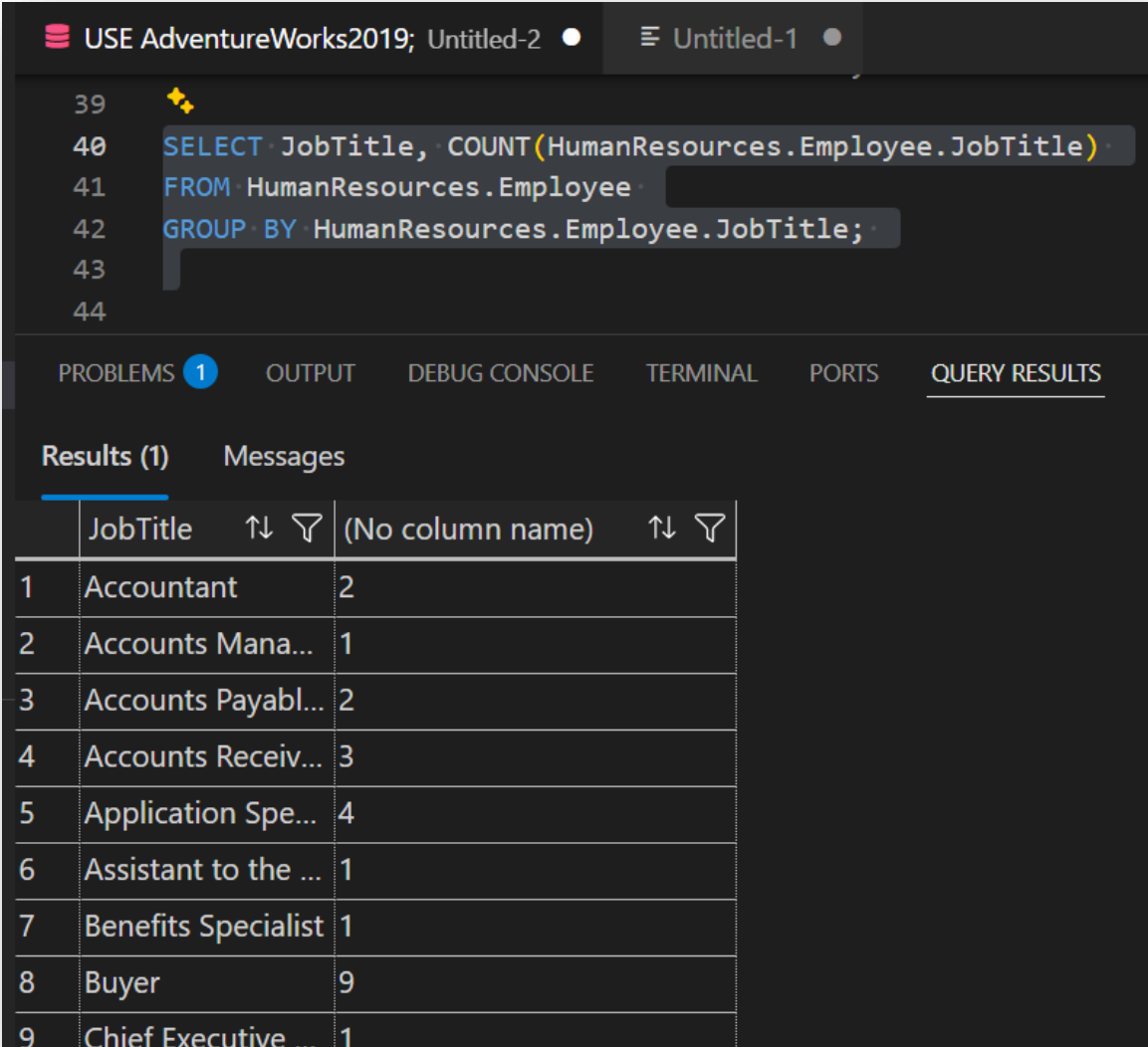
MIN(), MAX(), AVG(), STDEV()

3) SELECT QUERIES WITHOUT JOINING

In SQL, you can use the instruction “... GROUP BY attribute” to group the obtained results and even more you can specify “... HAVING condition” to filter with a condition.

How to count the number of employees per JobTitle?

```
SELECT JobTitle,
COUNT(HumanResources.Employee.JobTitle)
FROM HumanResources.Employee
GROUP BY
HumanResources.Employee.JobTitle;
```



The screenshot shows a SQL query window with the following code:

```
39
40 SELECT JobTitle, COUNT(HumanResources.Employee.JobTitle)
41 FROM HumanResources.Employee
42 GROUP BY HumanResources.Employee.JobTitle;
43
44
```

Below the query window, the 'QUERY RESULTS' tab is active, displaying the results of the query in a table format:

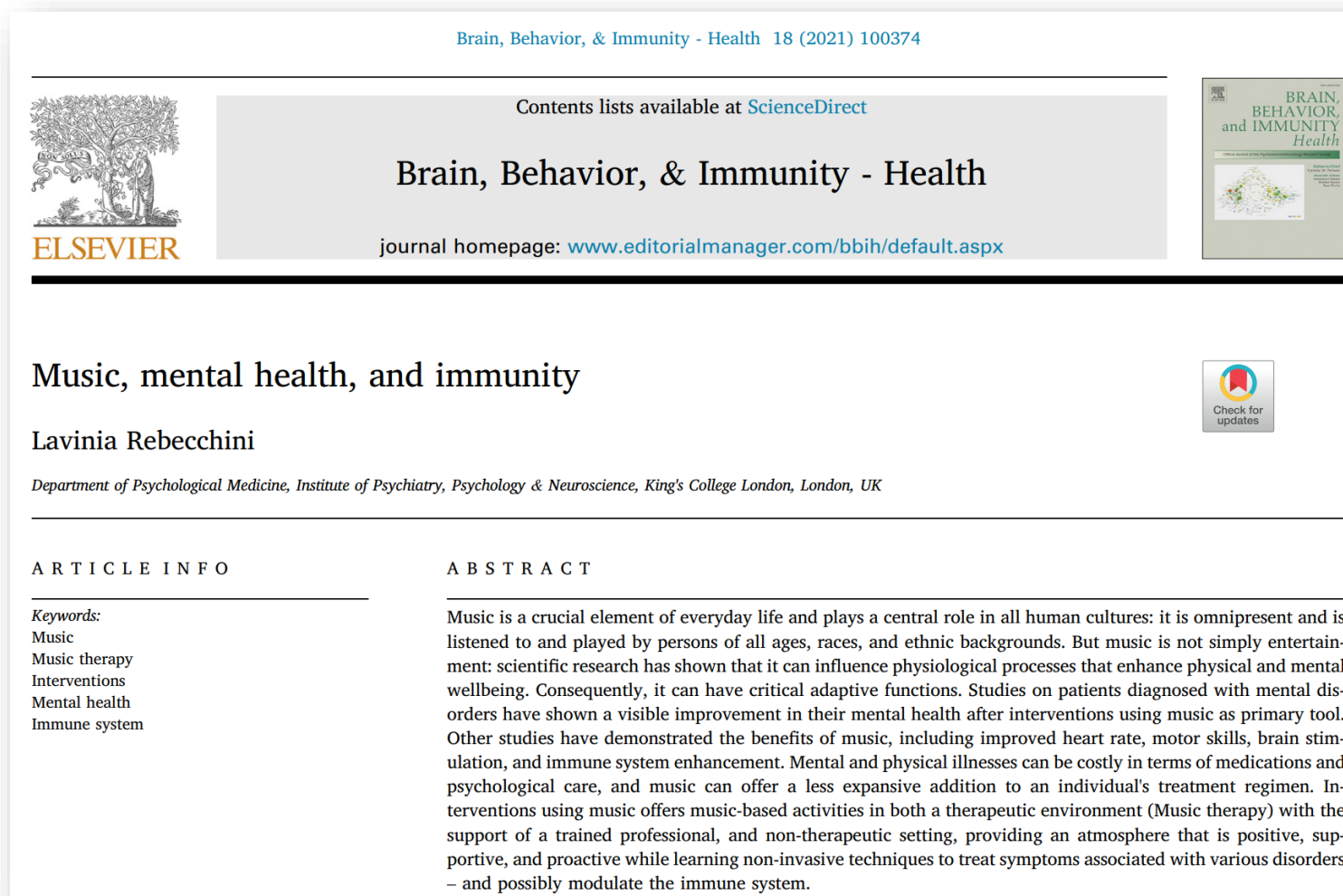
	JobTitle	(No column name)
1	Accountant	2
2	Accounts Mana...	1
3	Accounts Payabl...	2
4	Accounts Receiv...	3
5	Application Spe...	4
6	Assistant to the ...	1
7	Benefits Specialist	1
8	Buyer	9
9	Chief Executive ...	1

TIPS AND TRICKS

Be positive! B+

Music is good
for health, listen
to music!

Source: <https://pmc.ncbi.nlm.nih.gov/articles/PMC8566759/>



4) JOINING TABLES

- When the information you want to get is **across multiple tables**, we need to use a JOIN **between these tables to retrieve the data**
- Let's play with SELECT queries with JOINS!

4) JOINING TABLES

How joins work?

Cartesian product

You have two sets A and B
 $A=\{a1, a2, a3\}$ $B=\{b1, b2\}$

The cartesian product is
defined as:

$$A \times B = \{ (u,v), u \in A, v \in B \}$$

Compute $A \times B$:

$$A \times B = \{ (a1, b1), (a2, b1), (a3, b1), (a1, b2), (a2, b2), (a3, b2) \}$$

4) JOINING TABLES

Cartesian product

If we know the number of elements in the sets:

$\text{card}(A)=m$ and $\text{card}(B)=n$

What is the number of elements in $A \times B$, $\text{card}(A \times B)$?

$$\text{card}(A \times B) = m * n$$

4) JOINING TABLES

Cartesian product

How joins work? Tables as sets

You can mathematically consider tables as sets

Each tuple (row) of the table is an element of the set

4) JOINING TABLES

Cartesian product

Car

Id	plateNumber	brand	colorId
----	-------------	-------	---------

1	ABC 123	Volvo	2
---	---------	-------	---

1	CDE 345	Saab	3
---	---------	------	---

Color

Id	name
----	------

1	Red
---	-----

2	Blue
---	------

3	White
---	-------

Cartesian product

Car={ (1, 'ABC 123', 'Volvo', 2),
(2, 'CDE345', 'Saab', 3) }

Color={ (1, 'Red'),
(2, 'Blue'),
(3, 'White') }

4) JOINING TABLES

Cartesian product for tables

Car = { (1, 'ABC 123', 'Volvo', 2),
(2, 'CDE345', 'Saab', 3) }

Color = { (1, 'Red'),
(2, 'Blue'),
(3, 'White') }

Car x Color = ?

Car x Color = { ((1, 'ABC 123', 'Volvo', 2), (1, 'Red')), ((2, 'CDE345', 'Saab', 3), (1, 'Red')),
((1, 'ABC 123', 'Volvo', 2), (2, 'Blue')),
((2, 'CDE345', 'Saab', 3), (2, 'Blue')),
((1, 'ABC 123', 'Volvo', 2), (3, 'White')), ((2, 'CDE345', 'Saab', 3), (3, 'White')) }

4) JOINING TABLES

Get rid of internal parentheses:

Car x Color = {
(1, 'ABC 123', 'Volvo', 2, 1, 'Red'), (2, 'CDE345', 'Saab', 3, 1, 'Red')),
(1, 'ABC 123', 'Volvo', 2, 2, 'Blue'), (2, 'CDE345', 'Saab', 3, 2, 'Blue'),
(1, 'ABC 123', 'Volvo', 2, 3, 'White'), (2, 'CDE345', 'Saab', 3, 3, 'White') }

Why is it an interesting result?

4) JOINING TABLES

- Get rid of internal parentheses:

Car x Color = {
(1, 'ABC 123', 'Volvo', 2, 1, 'Red'), (2,
'CDE345', 'Saab', 3, 1, 'Red')),
(1, 'ABC 123', 'Volvo', 2, 2, 'Blue'), (2,
'CDE345', 'Saab', 3 , 2, 'Blue'),
(1, 'ABC 123', 'Volvo', 2, 3, 'White'), (2,
'CDE345', 'Saab', 3 , 3, 'White') }

```
SELECT * FROM Car  
INNER JOIN Color
```

- Why is it an interesting result ?

4) JOINING TABLES

- Get rid of internal parentheses:

Car x Color = {
(1, 'ABC 123', 'Volvo', 2, 1, 'Red'), (2,
'CDE345', 'Saab', 3, 1, 'Red')),
(1, 'ABC 123', 'Volvo', 2, 2, 'Blue'), (2,
'CDE345', 'Saab', 3, 2, 'Blue'),
(1, 'ABC 123', 'Volvo', 2, 3, 'White'), (2,
'CDE345', 'Saab', 3, 3, 'White') }

```
SELECT * FROM Car  
INNER JOIN Color  
ON Car.colorId = Color.id
```

- Why is it an interesting result ?

4) JOINING TABLES

- Get rid of internal parentheses:

Car x Color = {
(1, 'ABC 123', 'Volvo', 2, 1, 'Red'), (2,
'CDE345', 'Saab', 3, 1, 'Red')),
(1, 'ABC 123', 'Volvo', 2, 2, 'Blue'), (2,
'CDE345', 'Saab', 3, 2, 'Blue'),
(1, 'ABC 123', 'Volvo', 2, 3, 'White'), (2,
'CDE345', 'Saab', 3, 3, 'White') }

```
SELECT * FROM Car  
INNER JOIN Color  
ON Car.colorId = Color.id
```

- Why is it an interesting result ?

4) JOINING TABLES

- Get rid of internal parentheses:

Car x Color = {
(1, 'ABC 123', 'Volvo', 2, 1, 'Red'), (2, 'CDE345', 'Saab', 3, 1, 'Red')),
(1, 'ABC 123', 'Volvo', 2, 2, 'Blue'), (2, 'CDE345', 'Saab', 3, 2, 'Blue'),
(1, 'ABC 123', 'Volvo', 2, 3, 'White'), (2, 'CDE345', 'Saab', 3, 3, 'White') }

```
SELECT * FROM Car  
INNER JOIN Color  
ON Car.colorId = Color.id
```

- Why is it an interesting result ?

4) JOINING TABLES

- Get rid of internal parentheses:

Car x Color = {

(1, 'ABC 123', 'Volvo', 2, 1, 'Red'), (2, 'CDE345', 'Saab', 3, 1, 'Red')),

(1, 'ABC 123', 'Volvo', 2, 2, 'Blue'), (2, 'CDE345', 'Saab', 3, 2, 'Blue'),

(1, 'ABC 123', 'Volvo', 2, 3, 'White'), (2, 'CDE345', 'Saab', 3, 3, 'White') }

```
SELECT * FROM Car
INNER JOIN Color
ON Car.colorId = Color.id
```

- Why is it an interesting result ?

4) JOINING TABLES

- Get rid of internal parentheses:

Car x Color = {

~~(1, 'ABC 123', 'Volvo', 2, 1, 'Red'), (2, 'CDE345', 'Saab', 3, 1, 'Red')),~~

~~(1, 'ABC 123', 'Volvo', 2, 2, 'Blue'), (2, 'CDE345', 'Saab', 3, 2, 'Blue'),~~

~~(1, 'ABC 123', 'Volvo', 2, 3, 'White'), (2, 'CDE345', 'Saab', 3, 3, 'White') }~~

```
SELECT * FROM Car
INNER JOIN Color
ON Car.colorId = Color.id
```

- Why is it an interesting result ?

4) JOINING TABLES

- Get rid of internal parentheses:

Car x Color = {

~~(1, 'ABC 123', 'Volvo', 2, 1, 'Red'), (2, 'CDE345', 'Saab', 3, 1, 'Red')),~~

(1, 'ABC 123', 'Volvo', 2, 2, 'Blue'), {
~~2, 'CDE345', 'Saab', 3, 2, 'Blue'),~~

~~(1, 'ABC 123', 'Volvo', 2, 3, 'White'), (~~
2, 'CDE345', 'Saab', 3, 3, 'White') }

SELECT * FROM Car

INNER JOIN Color

ON Car.colorId = Color.id

(1, 'ABC 123', 'Volvo', 2, 2, 'Blue'),
(2, 'CDE345', 'Saab', 3, 3, 'White')

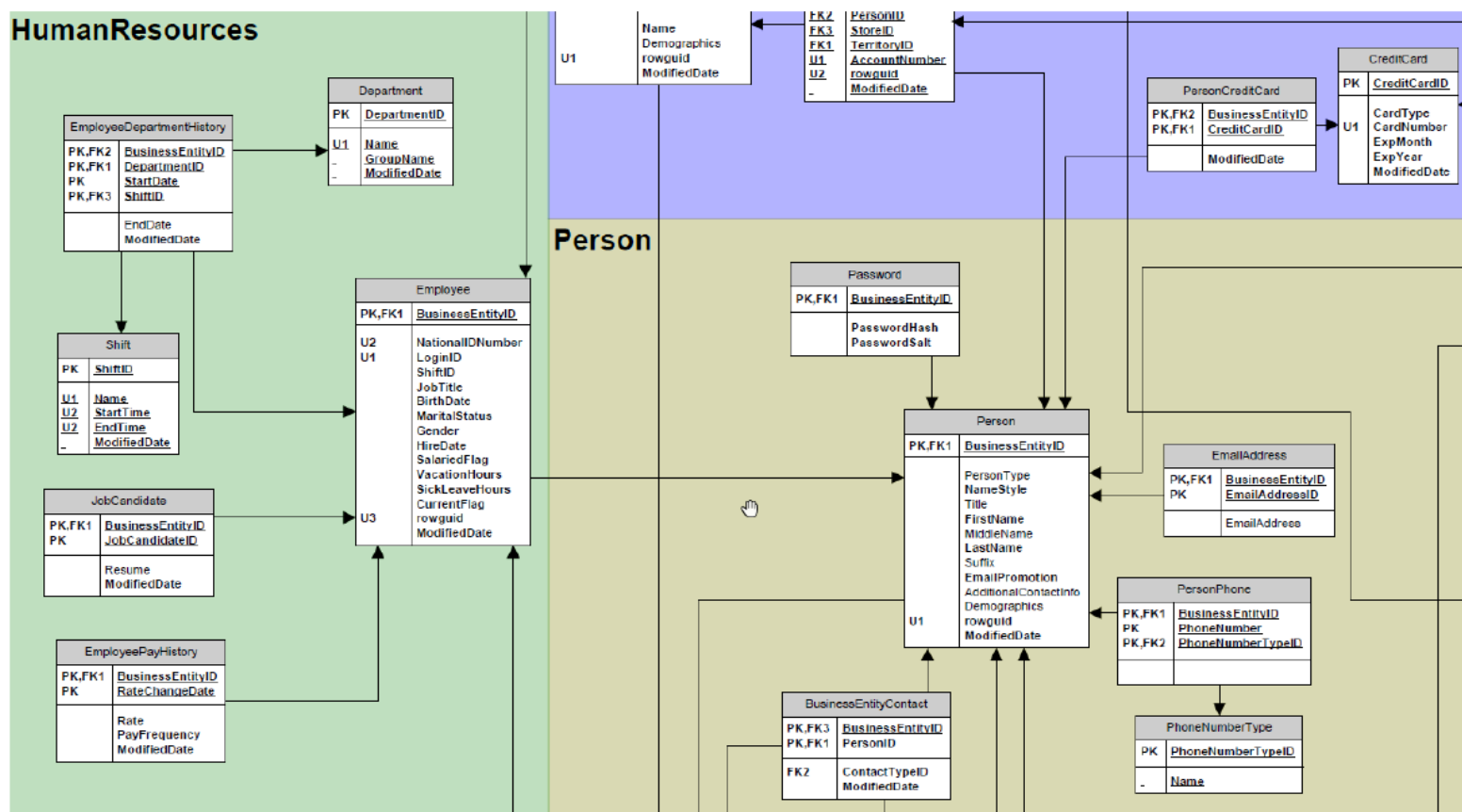
- Why is it an interesting result ?

4) JOINING TABLES

- What is the methodology for joining tables?
 - 1) Find in your tables the attributes you need to answer the question
 - 2) For each attribute found, you will need this table in your join
 - 3) Find the relations (common attributes) and the path between these tables
 - 4) Write the join in SQL following your path
- **Remark:** if all the needed attributes are in **one table ONLY**, there is **no need** for a **JOIN**!

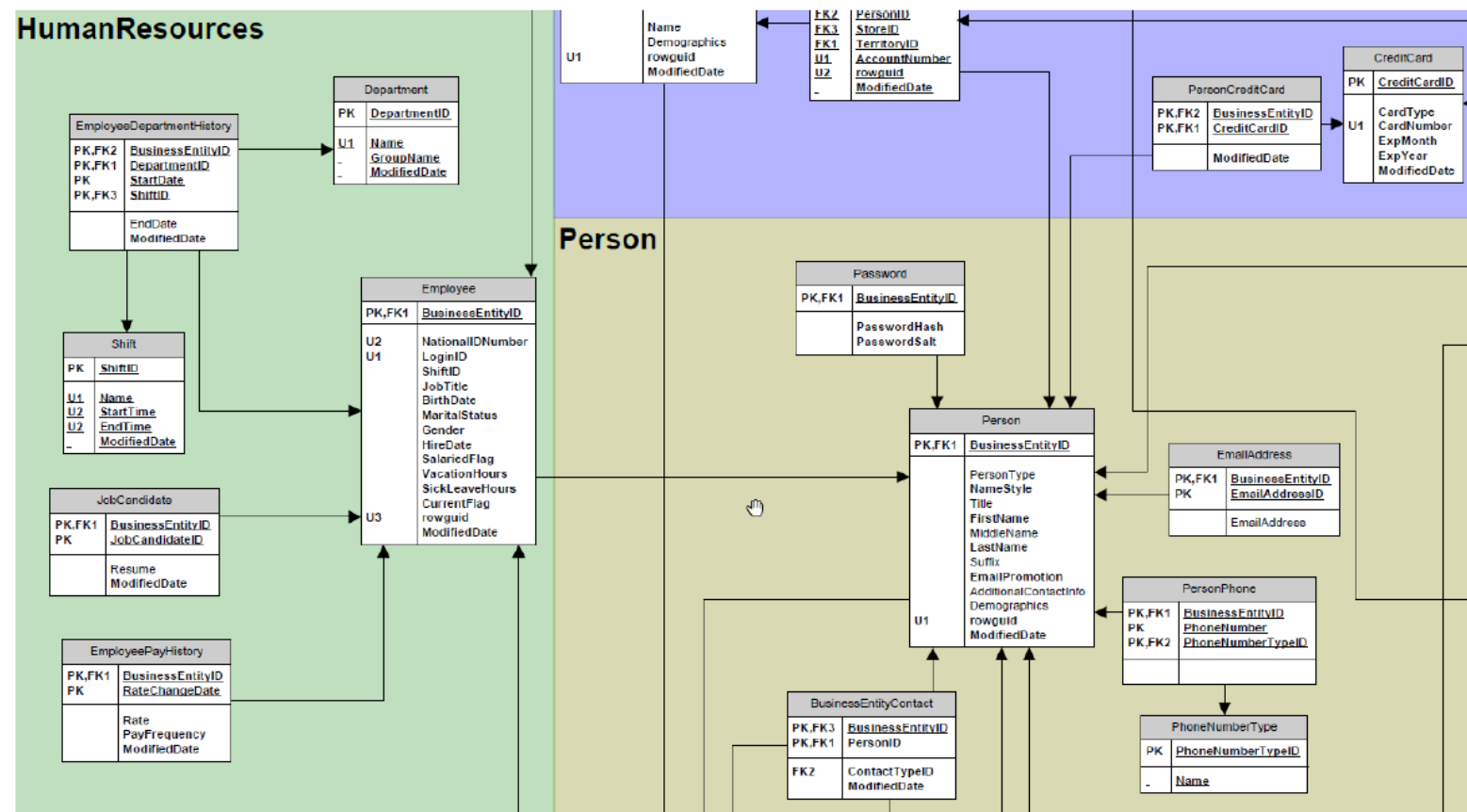
- 1) Find in your tables the attributes you need to answer the question
- 2) For each attribute found, you will need this table in your join
- 3) Find the relations (common attributes) and the path between these tables
- 4) Write the join in SQL following your path

- Find the
firstname,
lastname of the
employees that
are 'Database
Administrator'.



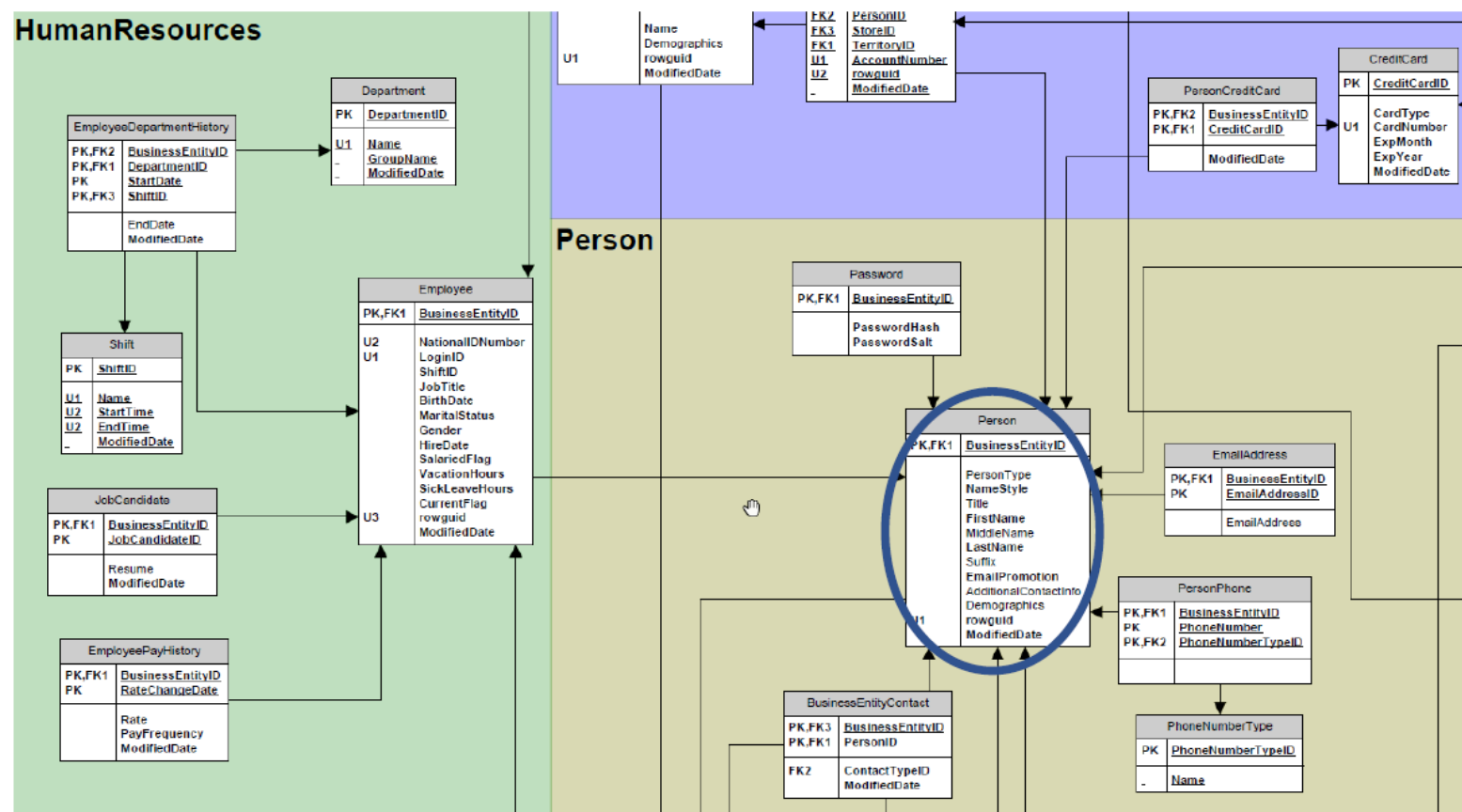
- 1) Find in your tables the attributes you need to answer the question
- 2) For each attribute found, you will need this table in your join
- 3) Find the relations (common attributes) and the path between these tables
- 4) Write the join in SQL following your path

- Find the **firstname**, **lastname** of the employees that are 'Database Administrator'.



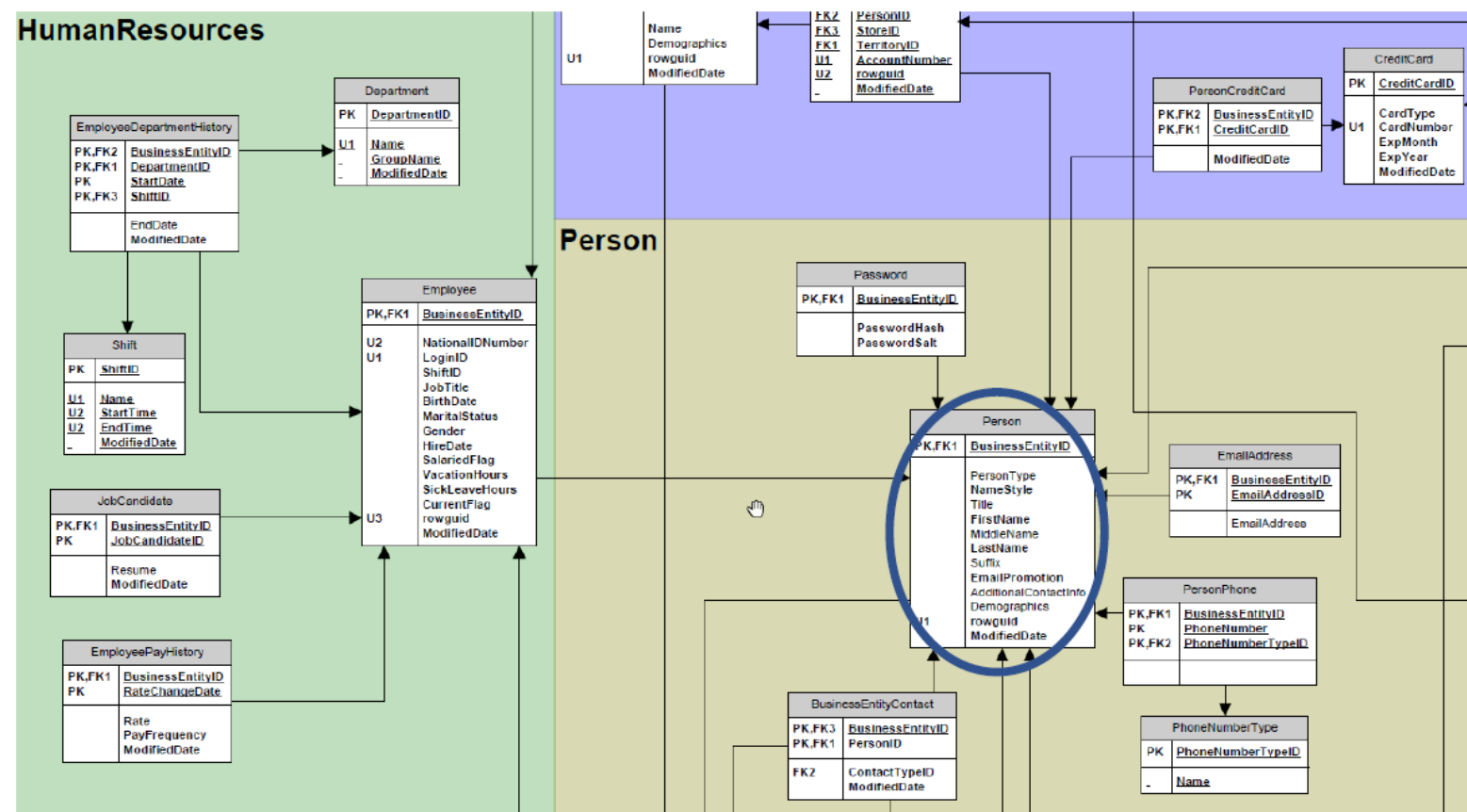
- 1) Find in your tables the attributes you need to answer the question
- 2) For each attribute found, you will need this table in your join
- 3) Find the relations (common attributes) and the path between these tables
- 4) Write the join in SQL following your path

- Find the **firstname**, **lastname** of the employees that are 'Database Administrator'.



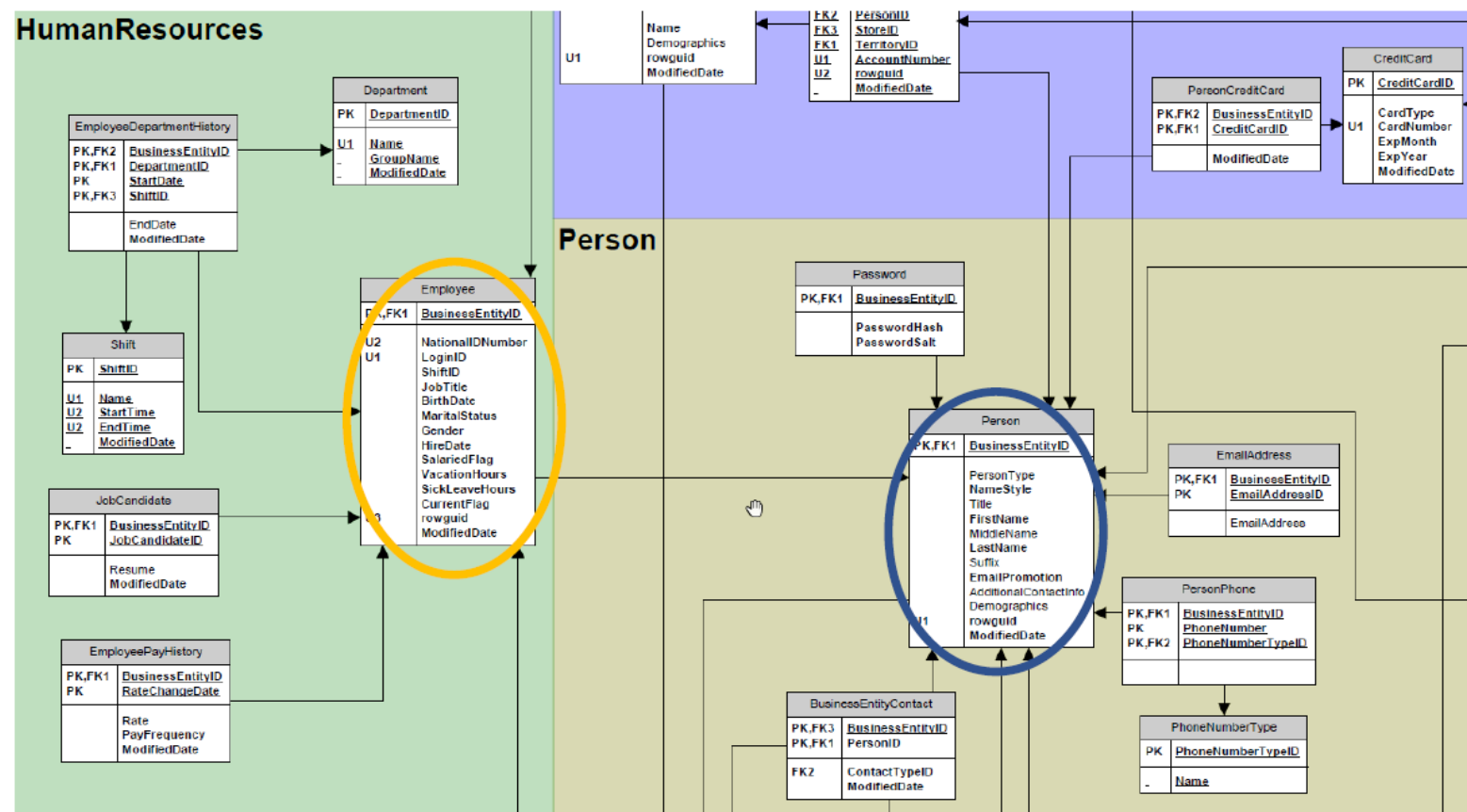
- 1) Find in your tables the attributes you need to answer the question
- 2) For each attribute found, you will need this table in your join
- 3) Find the relations (common attributes) and the path between these tables
- 4) Write the join in SQL following your path

- Find the **firstname**, **lastname** of the employees that are 'Database Administrator'.



- 1) Find in your tables the attributes you need to answer the question
- 2) For each attribute found, you will need this table in your join
- 3) Find the relations (common attributes) and the path between these tables
- 4) Write the join in SQL following your path

- Find the **firstname**, **lastname** of the employees that are 'Database Administrator'.



5) DIFFERENT TYPES OF JOINS

- Prompt asked to Gemini:

Can you generate an image (with white background) that explains visually what are JOINS in a relational database with a nice example of each possible JOIN?

5) DIFFERENT TYPES OF JOINS

- **Core Concept**
 - JOINS are used to combine data from two tables based on a common column. This image uses "Employees" and "Departments" tables connected by the "DeptID" column as a consistent example.
- **The Main JOIN Types**
 - This visual guide includes the three primary join types you requested:
 - LEFT JOIN: Selects all records from the "Employees" table and matching records from "Departments." If there's no match, NULL values are used for the "Departments" columns. This is ideal for finding data that might have a corresponding entry in the other table.
 - RIGHT JOIN: The inverse of the LEFT JOIN. It selects all records from the "Departments" table and matches from "Employees." For example, it lists all departments, even those without employees.
 - FULL JOIN: This returns all records when there is a match in either table. If there's a match, it combines them; if not, it fills with NULLs for the non-matching side.

The diagrams at the bottom show which subset of data is returned by each type of join, using consistent visual language across all examples.

IMPRESSIVE!

BUT FALSE: HALLUCINATIONS!

WHAT ARE JOINS IN A RELATIONAL DATABASE?

CORE CONCEPT

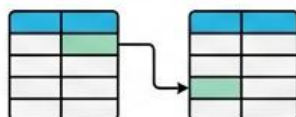


Table 1 EMPLOYEES

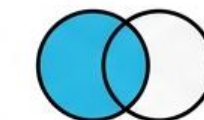
E_ID	E_NAME	DEPT_ID
1	ALICE	101
2	BOB	102
3	CHARLIE	104
4	DAVID	NULL

DEPARTMENTS

DEPT_ID	DEPT_NAME
101	HR
102	IT
103	SALES
104	MARKETING

LEFT JOIN

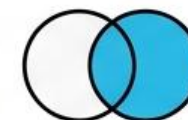
E_ID	E_NAME	DEPT_ID
1	ALICE	101
2	BOB	102
3	CHARLIE	104
4	DAVID	NULL
5	NULL	HR



Selects ALL records from the left table and matched records from the right.

RIGHT JOIN

E_ID	E_NAME	DEPT_ID
1	ALICE	101
2	BOB	102
3	CHARLIE	102
4	DAVID	NULL
5	NULL	SALES
6	IT	MARKETING

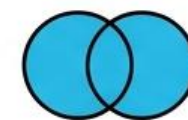


Selects ALL records from the right table and matched records from the left.

4 NULL 103

FULL JOIN

E_ID	E_NAME	DEPT_ID
1	ALICE	101
2	BOB	102
3	CHARLIE	104
4	DAVID	NULL
5	NULL	NULL
6	SALES	NULL

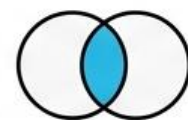


Selects ALL records when there is a match in EITHER table.

5 NULL 103

INNER JOIN

E_ID	E_NAME	DEPT_ID
1	ALICE	101
2	BOB	102
3	CHARLIE	104
4	DAVID	101



Selects records with matching values in BOTH tables.

Can you generate the SQL code to create these tables and test the different joins please?

```
=====
-- 1. Create the Database Schema (Tables) --
=====
-- Note: The schema matches the diagram's visual structure. -- The resulting
tables in the diagram often combine columns -- differently (like swapping
Dept_ID and Dept_Name), -- but these are the base definitions. CREATE TABLE
EMPLOYEES ( E_ID INT, E_NAME VARCHAR(50), DEPT_ID INT ); CREATE TABLE
DEPARTMENTS ( DEPT_ID INT, DEPT_NAME VARCHAR(50) ); --
=====
-- 2. Populate tables with sample data from the diagram --
=====
-- Populate 'Table 1: EMPLOYEES'
INSERT INTO EMPLOYEES (E_ID, E_NAME, DEPT_ID) VALUES (1, 'ALICE', 101), (2,
'BOB', 102), (3, 'CHARLIE', 104), (4, 'DAVID', NULL); -- NULL DEPT_ID
-- Populate 'DEPARTMENTS' table
INSERT INTO DEPARTMENTS (DEPT_ID, DEPT_NAME) VALUES (101, 'HR'), (102, 'IT'),
(103, 'SALES'), (104, 'MARKETING');
```

Can you generate the SQL code to create these tables and test the different joins please?

```
=====
-- 3. Perform the JOIN Operations to test results--
=====
-- Note: The visualization in the image is a conceptual model.
-- The resulting columns in the visual tables (e.g., placing DEPT_NAME first)
-- are rearranged here to match the image's specific visual output.
```

Example 1: INNER JOIN

-- Result match: Yes

----- Textbox: Selects
records with matching values in BOTH tables.

-- The image result includes E_ID, E_NAME, DEPT_NAME, and DEPT_ID.

```
SELECT      e.E_ID,      e.E_NAME,      d.DEPT_NAME,      d.DEPT_ID FROM      EMPLOYEES
eINNER JOIN  DEPARTMENTS d ON e.DEPT_ID = d.DEPT_ID;
```

...

LEFT JOIN

```
72
73 SELECT
74     e.E_ID,
75     e.E_NAME,
76     d.DEPT_NAME,
77     d.DEPT_ID
78 FROM
79     EMPLOYEES e
80 LEFT JOIN
81     DEPARTMENTS d ON e.DEPT_ID = d.DEPT_ID;
82
```

E_ID	E_NAME	DEPT_NAME	DEPT_ID
1	ALICE	HR	101
2	BOB	IT	102
3	CHARLIE	MARKETING	104
4	DAVID	NULL	NULL

RIGHT JOIN

```

87 -- We will follow the logical standard definition described.
88 -----
89 -- Textbox: Selects ALL records from the right table and matched records from the left.
90 -- This will return all Departments, and matching Employees. Sales will have NULL employees.
91
92 SELECT
93     e.E_ID,
94     e.E_NAME,
95     d.DEPT_NAME,
96     d.DEPT_ID
97 FROM
98     EMPLOYEES e
99 RIGHT JOIN

```

E_ID	E_NAME	DEPT_NAME	DEPT_ID
1	ALICE	HR	101
2	BOB	IT	102
3	CHARLIE	MARKETING	104
NULL	NULL	SALES	103

FULL JOIN

```

125
126 SELECT e.E_ID, e.E_NAME, d.DEPT_NAME, d.DEPT_ID
127 FROM EMPLOYEES e
128 LEFT JOIN DEPARTMENTS d ON e.DEPT_ID = d.DEPT_ID
129 UNION
130 SELECT e.E_ID, e.E_NAME, d.DEPT_NAME, d.DEPT_ID
131 FROM EMPLOYEES e
132 RIGHT JOIN DEPARTMENTS d ON e.DEPT_ID = d.DEPT_ID;

```

E_ID	E_NAME	DEPT_NAME	DEPT_ID
NULL	NULL	SALES	103
1	ALICE	HR	101
2	BOB	IT	102
3	CHARLIE	MARKETING	104
4	DAVID	NULL	NULL

INNER JOIN

```
53
54 SELECT
55     e.E_ID,
56     e.E_NAME,
57     d.DEPT_NAME,
58     d.DEPT_ID
59 FROM
60     EMPLOYEES e
61 INNER JOIN
62     DEPARTMENTS d ON e.DEPT_ID = d.DEPT_ID;
63
```

E_ID	E_NAME	DEPT_NAME	DEPT_ID
1	ALICE	HR	101
2	BOB	IT	102
3	CHARLIE	MARKETING	104

Exercises:

We will use the "AdventureWorks2019" database

- 1) Find the firstname, lastname of the employees that are 'Database Administrator'.
- 2) Find the department name, firstname, lastname of the employees that are 'Database Administrator'.
- 3) Find the department name, phone number, firstname, lastname of the employees that are 'Database Administrator'.
- 4) Find the department name, phone number, firstname, lastname, credit card number of the employees that are 'Database Administrator'.
- 5) Display the firstname, lastname, product name, credit card number and order date of the persons who ordered a "Sport-100 Helmet, Red" from the Sales department.

ANY QUESTIONS?